stv_v2

Generated by Doxygen 1.9.8

Chapter 1

STV2 - StraTegic Verifier version 2

1.1 Usage

To run:

```
cd build
make clean
make
./stv
```

or

```
cd build
./build-run
```

Configuration file:

```
build/config.txt
```

CLI configuration overwrite:

```
# Input model
./stv --file PATH_TO_MODEL
./stv -f PATH_TO_MODEL
# Mode
./stv -m 0  #
./stv -m 1  # generate GlobalModel
./stv -m 2  # run verification
./stv -m 3  # same as 1 && 2
# Flags
# --OUTPUT_GLOBAL_MODEL      stdout data on global model (after expandAllStates)
# --OUTPUT_LOCAL_MODELS      stdout data on local models ()
# --OUTPUT_DOT_FILES         generate .dot files for agent templates, local and global models
# --ADD_EPSILON_TRANSITIONS  generate global models with epsilon transitions
# --OVERWRITE_FORMULA        replace the formula from the model file with a different one
# --COUNTEREXAMPLE           output counterexample path if the formula verification returns an ERR
# --REDUCE                   reduce the amount of states and transitions using a DFS-POR algorithm and
      select the first correct transition
# --REDUCE_ALL               reduce the amount of states and transitions using a DFS-POR algorithm and
      select all avaliable transitions
# --FIXPOINT                 enables fixpoint approximation
# --NATURAL_STRATEGY         generate a natural strategy for the given model
# --STRATEGY_FROM_FILE       set a strategy to the one given in a file
```

1.2 Tests

To run tests:

```
cd build
make clean
make sample_test
./sample_test
```

or
```
cd build
./build-test
```

To run larger tests:
```
cd build
make clean
make sample_test
./sample_test
```

or
```
cd build
./build-big-test
```

You might need to run
```
ulimit -s unlimited
```

beforehand

1.3 Performance estimation

Ubuntu/WSL:

```
# Minimal
> /usr/bin/time -f "%M\t%e" ./stv
# %M - maximum resident set size in KB
# %e - elapsed real time (wall clock) in seconds

# More detailed
> /usr/bin/time -f "time result\ncmd:%C\nreal %es\nuser %Us \nsys  %Ss \nmemory:%MKB \ncpu %P" ./stv
# %C   command line and arguments
# %e   elapsed real time (wall clock) in seconds
# %U   user time in seconds
# %S   system (kernel) time in seconds
# %M   maximum resident set size in KB
# %P   percent of CPU this job got

# Full (verbose)
> /usr/bin/time -v ./stv
```

1.4 Specification

The specification language was inspired by ISPL (Interpreted Systems Programming Language) from MCMAS. The detailed syntax for the input format can be derived from ∗./src/reader/{parser.y,scanner.l}∗, which intrinsically make up an EBNF grammar.
For the most parts, it is simple enough to get intuition just from looking at example's source code and the program's output.

IMPORTANTS NOTES:

1. the (local) action names must be unique;
2. the transition relation (from the global model) should be serial;

1.5 Examples and templates

In ∗./examples∗ and ∗./tests/examples∗ there are several ready-to-use MAS specification files together with a proposed property (captured by ATL formula) for verification.

Often, we would want to reason about different (data-)configurations of the same system.
Using the templates we can parameterize the system specification, such that we only need to describe its dynamic behavior.
A template can be fed with a configuration data to generate a concrete instance of a system.
Moreover, their use is independent from the tool: one can choose any templating engine (of the myriads available) or even write a custom one from the scratch.

Here, we utilize the `EJS` templating engine. It has a CLI support, which is comes in handy for the tests/benchmarks that involve systems in multiple configurations.

```
# EJS feeds the data (as a list of key:val pairs) to the template file to generate the output:
> npm exec -- ejs TEMPLATE_FILE.ejs -i "{PARAM1:VAL1,PARAM2:VAL2,...}" -o OUTPUT_FILE.txt


# Possible generation query for the "trains":
> npm exec -- ejs Trains.ejs -i '{"N_TRAINS":3,"WITH_FORMULA":1}' -o 3Trains1Controller.txt

# Possible generation query for the "simple voting":
> npm exec -- ejs Simple_voting.ejs -i '{"N_VOTERS":2,"N_CANDIDATES":1,"WITH_FORMULA":0}' -o
      2Voters1Coercer1Candidate.txt
```

1.6 Misc

With `OUTPUT_DOT_FILES` flag the program outputs ∗.dot∗ files for templates, local and global models where:

- nodes are labelled with its location name (comma-separated for the global state)
- shared transitions are denoted by blue colour

Use Graphviz `(link)` to view in other format (eps, pdf, jpeg, etc.):

```
# Analogously for other formats
dot -Tpng lts_of_AGENT.dot > lts_of_AGENT.png
```

For the smaller graphs use *dot2png.sh* script, which converts all ∗.dot∗ files from a current folder to ∗.png∗.
For bigger ones use `svg` format (may be viewed in Inkscape) and *dot2svg.sh*.

1.7 Web interface

The web interface is available at `http://stv.cs-htiew.com/`.

1.8 Credits

Lead developer: Mateusz Kamiński (@Mathisplay)

Project supervisor: Damian Kurpiewski (@blackbat13)

Initial codebase: Witold Pazderski

Additional features: Yan Kim

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 5

Class Documentation

5.1 Action Struct Reference

Single action template for probabilistic verification.

`#include <TypesDependency.hpp>`

Public Attributes

- vector< string > * **states**
- string **hash**
- string **actionName**

5.1.1 Detailed Description

Single action template for probabilistic verification.

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.2 ActionTemplate Class Reference

Represents a selected action.

`#include <strategyNodes.hpp>`

Public Member Functions

- ActionTemplate (vector< string > *_states, string _actionName)

  Constructor for the ActionTemplate.

Public Attributes

- vector< string > * **states**

  Vector of states from which the action has to be executed from.
- string **hash**

  Contains the states in a.
- string **actionName**

5.2.1 Detailed Description

Represents a selected action.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 ActionTemplate()

```
ActionTemplate::ActionTemplate (
            vector< string > * _states,
            string _actionName )
```

Constructor for the ActionTemplate.

Constructor for an ActionTemplate.

Parameters

_states	Vector of states from which the action has to be executed from.
_actionName	Action executed from the aforementioned states.

The documentation for this class was generated from the following files:

- strategyNodes.hpp
- strategyNodes.cc

5.3 Agent Class Reference

Contains all data for a single Agent, including id, name and all of the agents' variables.

```
#include <Agent.hpp>
```

Public Member Functions

- Agent (int _id, string _name)

  Constructor for the Agent class, assigning it an id and name.
- LocalState * includesState (LocalState *state)

  Checks if there is an equivalent LocalState in the model to the one passed as an argment.

Public Attributes

- int **id**

  Identifier of the agent.
- string **name**

  Name of the agent.
- set< Var * > **vars**

  Variable names for the agent.
- LocalState * **initState**

  Initial state of the agent.
- vector< LocalState * > **localStates**

  Local states for this agent.
- vector< LocalTransition * > **localTransitions**

  Local transitions for this agent.

5.3.1 Detailed Description

Contains all data for a single Agent, including id, name and all of the agents' variables.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 Agent()

```
Agent::Agent (
            int _id,
            string _name )  [inline]
```

Constructor for the Agent class, assigning it an id and name.

Parameters

_id	Identifier of the new agent.
_name	Name of the new agent.

5.3.3 Member Function Documentation

5.3.3.1 includesState()

```
LocalState * Agent::includesState (
            LocalState * state )
```

Checks if there is an equivalent LocalState in the model to the one passed as an argment.

Parameters

state	A pointer to LocalState to be checked.

Returns

Returns a pointer to an equivalent LocalState if such exists, otherwise returns NULL.

The documentation for this class was generated from the following files:

- Agent.hpp
- Agent.cpp

5.4 AgentTemplate Class Reference

Represents a single agent loaded from the description from a file.

`#include <nodes.hpp>`

Public Member Functions

- **AgentTemplate** ()
  Constructor for an AgentTemplate.
- virtual AgentTemplate & setIdent (string _ident)
  Set the identifier of an agent.
- virtual AgentTemplate & setInitState (string _startState)
  Set the initial state of the agent.
- virtual AgentTemplate & addLocal (set< string > *variables)
  Adds local variables to an agent.
- virtual AgentTemplate & addPersistent (set< string > *variables)
  Adds persistent variables to an agent.
- virtual AgentTemplate & addInitial (set< Assignment * > *assigns)
  Adds initial assignments.
- virtual AgentTemplate & addTransition (TransitionTemplate *_transition)
  Adds a transition to the agent.
- virtual Agent * generateAgent (int id)
  Generate a new agent for the model.

Friends

- class **DotGraph**

5.4.1 Detailed Description

Represents a single agent loaded from the description from a file.

5.4.2 Member Function Documentation

5.4.2.1 addInitial()

```
AgentTemplate & AgentTemplate::addInitial (
            set< Assignment * > * assigns )  [virtual]
```

Adds initial assignments.

Sets initial values of agent's variables.

Parameters

assigns	Assignments to be added.

Returns

Returns a pointer to self.

Parameters

assigns	Set of variables to assign.

Returns

Returns itself.

5.4.2.2 addLocal()

```
AgentTemplate & AgentTemplate::addLocal (
            set< string > * variables )  [virtual]
```

Adds local variables to an agent.

Adds local variables to the agent.

Parameters

variables	Set of variables to be added.

Returns

Returns a pointer to self.

Parameters

variables	A pointer to a set of strings with the variables to be added.

Returns

Returns itself.

5.4.2.3 addPersistent()

```
AgentTemplate & AgentTemplate::addPersistent (
            set< string > * variables )  [virtual]
```

Adds persistent variables to an agent.

Adds persistent variables to the agent.

Parameters

variables	Set of variables to be added.

Returns

Returns a pointer to self.

Parameters

variables	A pointer to a set of strings with the variables to be added.

Returns

Returns itself.

5.4.2.4 addTransition()

```
AgentTemplate & AgentTemplate::addTransition (
            TransitionTemplate * _transition )  [virtual]
```

Adds a transition to the agent.

Adds a transition for the agent.

Parameters

_transition	Transition to be added.

Returns

Returns a pointer to self.

Parameters

_transition	Transition to be added.

Returns

Returns itself.

5.4.2.5 generateAgent()

```
Agent * AgentTemplate::generateAgent (
            int id )  [virtual]
```

Generate a new agent for the model.

Generates an agent for the model.

Parameters

id	Identification number defining a new Agent.

Returns

Returns a pointer to a new Agent.

Parameters

id	Identifier of the new Agent.

Returns

Returns a pointer to a newly created Agent.

5.4.2.6 setIdent()

```
AgentTemplate & AgentTemplate::setIdent (
            string _ident )  [virtual]
```

Set the identifier of an agent.

Sets the identifier of an agent.

Parameters

_ident	New agent identifier.

Returns

Returns a pointer to self.

Parameters

_ident	String with a new identifier.

Returns

Returns itself.

5.4.2.7 setInitState()

```
AgentTemplate & AgentTemplate::setInitState (
            string _initState )  [virtual]
```

Set the initial state of the agent.

Sets initial state of an agent.

Parameters

_startState	New inital agent state.

Returns

Returns a pointer to self.

Parameters

_initState	String with a new state.

Returns

Returns itself.

The documentation for this class was generated from the following files:

- nodes.hpp
- nodes.cc

5.5 Assignment Class Reference

Represents an assingment.

`#include <nodes.hpp>`

Public Member Functions

- Assignment (string _ident, ExprNode *_exp)
  Constructor for an Assignment class.
- virtual void assign (Environment &env)
  Make an assignment in a given environment.

Public Attributes

- string **ident**
  To what we should assign a value.
- ExprNode * **value**
  A value to be assigned.

5.5.1 Detailed Description

Represents an assingment.

5.5.2 Constructor & Destructor Documentation

5.5.2.1 Assignment()

```
Assignment::Assignment (
            string _ident,
            ExprNode * _exp )  [inline]
```

Constructor for an Assignment class.

Parameters

_ident	To what we should assign a value.
_exp	A value to be assigned.

5.5.3 Member Function Documentation

5.5.3.1 assign()

```
virtual void Assignment::assign (
            Environment & env )  [inline], [virtual]
```

Make an assignment in a given environment.

Parameters

env	Environment in which to make an assignment.

The documentation for this class was generated from the following file:

- nodes.hpp

5.6 Cfg Struct Reference

Public Attributes

- std::string **fname**

  path to input file with system specification
- int **stv_mode**

  stv_code as sum/combination of (1 - expandAllStates, 2 - verify, 4 - print metadata, 8 - run experiments)
- bool **output_local_models**

  (obsolete) print data on local model
- bool **output_global_model**

  (obsolete) print data on local model
- bool **output_dot_files**

  flag for .dot export (by default exports templates and local/global models)
- std::string **dotdir**

  pathprefix for .dot files export
- int **model_id**
- bool **add_epsilon_transitions**

  add epsilon transitions to the states in the model when it's blocked for some reason
- bool **formula_from_parameter**
- std::string **formula**
- bool **counterexample**
- bool **reduce**
- bool **reduce_all**
- std::string **reduce_args**

- bool **fixpoint**
- bool **natural_strategy**
- bool **probability** = false
- bool **verify_strategy**
- std::string **strategy_file_path**

The documentation for this struct was generated from the following file:

- Common.hpp

5.7 Condition Struct Reference

Represents a condition for LocalTransition.

`#include <Types.hpp>`

Public Attributes

- Var * **var**

  Pointer to a variable.
- ConditionOperator **conditionOperator**

  Conditional operator for the variable.
- int **comparedValue**

  Condition value to be met.

5.7.1 Detailed Description

Represents a condition for LocalTransition.

The documentation for this struct was generated from the following file:

- Types.hpp

5.8 DecisionEntry Struct Reference

Public Member Functions

- string **toString** ()

Public Attributes

- HistoryEntryType **type** = HistoryEntryType::CONTEXT

  Type of the history record.
- GlobalState * **globalState** = nullptr

  Saved global state.
- GlobalTransition * **decision** = nullptr

  Selected transition.
- bool **globalTransitionControlled** = false

  Is the transition controlled by an agent in coalition.
- GlobalStateVerificationStatus **prevStatus** = GlobalStateVerificationStatus::UNVERIFIED

  Previous model verification state.
- GlobalStateVerificationStatus **newStatus** = GlobalStateVerificationStatus::UNVERIFIED

  Next model verification state.
- int **depth** = 0

  Iteration depth.
- StrategyEntry **strategy** = StrategyEntry()

  Holds currently processed strategy for the current state.
- float **stateProbabilityPrevious** = 0.0

  Holds previous GlobalState probability.
- ProbabilityCalculationType **previousProbabilityEntryType** = ProbabilityCalculationType::NONE_↩ PROBABILITY

  Holds previous probability answer calculation method.
- ProbabilityCalculationType **activeProbabilityEntryType** = ProbabilityCalculationType::SUM_PROBABILITY

  Holds currently used probability calculation method, used to rollback correct probability value.
- ProbabilityTrueFalse **previousProbabilityAnswerValues**

  Holds previous true/false probability.

The documentation for this struct was generated from the following file:

- VerificationIterative.hpp

5.9 DotGraph Class Reference

Public Member Functions

- **DotGraph** ()

  empty graph
- **DotGraph** (GlobalModel *const gm, bool extended=false, bool correct=false)

  parses the transitions/states templates into nodes/edges
- **DotGraph** (Agent *const ag, bool extended=false)

  parses the local transitions/states templates into nodes/edges
- **DotGraph** (AgentTemplate *const at)

  parses the edge/state templates into nodes/edges
- void saveToFile (std::string pathprefix, std::string nameprefix, std::string basename="")

  creates a `.dot` file

Public Attributes

- std::vector< std::string > **nodes**
- std::vector< std::string > **edges**
- std::string **graphName**
- DotGraphBase **graphBase**

Static Public Attributes

- static string styleString

  (temporary hard-coded) graphviz visual configuration

Protected Member Functions

- void addNode (std::string id, std::string name)

  creates a node string in graphviz syntax
- void addEdge (std::string src, std::string trg, std::string label)

  creates an edge string in graphviz syntax

5.9.1 Member Function Documentation

5.9.1.1 addEdge()

```
void DotGraph::addEdge (
            std::string src,
            std::string trg,
            std::string label )  [protected]
```

creates an edge string in graphviz syntax

Parameters

src	id of a source node
trg	id of a target node
label	edge label and, possibly, extra attributes

5.9.1.2 addNode()

```
void DotGraph::addNode (
            std::string id,
            std::string name )  [protected]
```

creates a node string in graphviz syntax

Parameters

id	unique internal node identifier
name	displayed node label/name

5.9.1.3 saveToFile()

```
void DotGraph::saveToFile (
            std::string pathprefix,
            std::string nameprefix,
            std::string basename = "" )
```

creates a `.dot` file

Parameters

basename	name of file (parent graph name if blank)

5.9.2 Member Data Documentation

5.9.2.1 styleString

```
std::string DotGraph::styleString  [static]
```

Initial value:

```
=
    "\tedge[fontsize=\"10\"]\n"
    "\tnode [\n"
    "\t\tshape=circle,\n"

    "\t\twidth=auto,\n"
    "\t\tcolor=\"black\",\n"
    "\t\tfillcolor=\"#eeeeee\",\n"
    "\t\tstyle=\"filled,solid\",\n"
    "\t\tfontsize=8,\n"
    "\t\tfontname=\"Roboto\"\n"
    "\t]\n"
    "\tfontname=Consolas\n"
    "\tlayout=dot\n"
```

(temporary hard-coded) graphviz visual configuration

The documentation for this class was generated from the following files:

- DotGraph.hpp
- DotGraph.cpp

5.10 EpistemicClass Struct Reference

Represents a single epistemic class.

```
#include <EpistemicClass.hpp>
```

Public Attributes

- string **hash**

  Hash of that epistemic class.
- map< string, GlobalState * > **globalStates**

  Map of GlobalState hashes to according GlobalState pointers bound to this epistemic class.
- GlobalTransition * **fixedCoalitionTransition**

  Transition that was already selected in this epistemic class. Model has to choose this transition if it is already set.

5.10.1 Detailed Description

Represents a single epistemic class.

The documentation for this struct was generated from the following file:

- EpistemicClass.hpp

5.11 ExprAdd Class Reference

Node for addition.

`#include <expressions.hpp>`

Public Member Functions

- ExprAdd (ExprNode *_larg, ExprNode *_rarg)
  Addition expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.11.1 Detailed Description

Node for addition.

5.11.2 Constructor & Destructor Documentation

5.11.2.1 ExprAdd()

```
ExprAdd::ExprAdd (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Addition expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.11.3 Member Function Documentation

5.11.3.1 eval() [1/2]

```
int ExprAdd::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.11.3.2 eval() [2/2]

```
int ExprAdd::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.12 ExprAnd Class Reference

Node for AND operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprAnd (ExprNode *_larg, ExprNode *_rarg)
  Logic AND expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.12.1 Detailed Description

Node for AND operator.

5.12.2 Constructor & Destructor Documentation

5.12.2.1 ExprAnd()

```
ExprAnd::ExprAnd (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Logic AND expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.12.3 Member Function Documentation

5.12.3.1 eval() [1/2]

```
int ExprAnd::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.12.3.2 eval() [2/2]

```
int ExprAnd::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.13 ExprConst Class Reference

Node for a constant.

`#include <expressions.hpp>`

Public Member Functions

- ExprConst (int _val)

  Constant expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int eval (Environment &env)

5.13.1 Detailed Description

Node for a constant.

5.13.2 Constructor & Destructor Documentation

5.13.2.1 ExprConst()

```
ExprConst::ExprConst (
            int _val )  [inline]
```

Constant expression constructor.

Parameters

_val	ExprConst value.

5.13.3 Member Function Documentation

5.13.3.1 eval() [1/2]

```
int ExprConst::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.13.3.2 eval() [2/2]

```
int ExprConst::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.14 ExprDiv Class Reference

Node for division.

`#include <expressions.hpp>`

Public Member Functions

- ExprDiv (ExprNode *_larg, ExprNode *_rarg)
  Division expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.14.1 Detailed Description

Node for division.

5.14.2 Constructor & Destructor Documentation

5.14.2.1 ExprDiv()

```
ExprDiv::ExprDiv (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Division expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.14.3 Member Function Documentation

5.14.3.1 eval() [1/2]

```
int ExprDiv::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.14.3.2 eval() [2/2]

```
int ExprDiv::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.15 ExprEq Class Reference

Node for "==" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprEq (ExprNode *_larg, ExprNode *_rarg)

  Equals expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int eval (Environment &env)

5.15.1 Detailed Description

Node for "==" operator.

5.15.2 Constructor & Destructor Documentation

5.15.2.1 ExprEq()

```
ExprEq::ExprEq (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Equals expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.15.3 Member Function Documentation

5.15.3.1 eval() [1/2]

```
int ExprEq::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.15.3.2 eval() [2/2]

```
int ExprEq::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.16 ExprGe Class Reference

Node for ">=" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprGe (ExprNode *_larg, ExprNode *_rarg)
  Greater or equal expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.16.1 Detailed Description

Node for ">=" operator.

5.16.2 Constructor & Destructor Documentation

5.16.2.1 ExprGe()

```
ExprGe::ExprGe (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Greater or equal expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.16.3 Member Function Documentation

5.16.3.1 eval() [1/2]

```
int ExprGe::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.16.3.2 eval() [2/2]

```
int ExprGe::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.17 ExprGt Class Reference

Node for ">" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprGt (ExprNode *_larg, ExprNode *_rarg)
  Greater than expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.17.1 Detailed Description

Node for ">" operator.

5.17.2 Constructor & Destructor Documentation

5.17.2.1 ExprGt()

```
ExprGt::ExprGt (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Greater than expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.17.3 Member Function Documentation

5.17.3.1 eval() [1/2]

```
int ExprGt::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.17.3.2 eval() [2/2]

```
int ExprGt::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.18 ExprHart Class Reference

Node for a Hartley operator.

`#include <expressions.hpp>`

Public Member Functions

- ExprHart (string _agentName, bool _le, int _val, vector< ExprNode * > *_arg)
  Constant expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.18.1 Detailed Description

Node for a Hartley operator.

5.18.2 Constructor & Destructor Documentation

5.18.2.1 ExprHart()

```
ExprHart::ExprHart (
            string _agentName,
            bool _le,
            int _val,
            vector< ExprNode * > * _arg )  [inline]
```

Constant expression constructor.

Parameters

_agentName	Agent name for Hartley operator.
_le	Flag for if Hartley coefficient should be less equal or greater equal.
_val	Number to compare the result to.
_arg	Expression to verify with the given knowledge.

5.18.3 Member Function Documentation

5.18.3.1 eval() [1/2]

```
int ExprHart::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.18.3.2 eval() [2/2]

```
int ExprHart::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.19 ExprIdent Class Reference

Node for an identifier.

`#include <expressions.hpp>`

Public Member Functions

- ExprIdent (string _ident)
  Identifier expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator ∗generator, GlobalState ∗globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.19.1 Detailed Description

Node for an identifier.

5.19.2 Constructor & Destructor Documentation

5.19.2.1 ExprIdent()

```
ExprIdent::ExprIdent (
            string _ident )  [inline]
```

Identifier expression constructor.

Parameters

_ident	ExprIdent value.

5.19.3 Member Function Documentation

5.19.3.1 eval() [1/2]

```
int ExprIdent::eval (
            Environment & env )  [virtual]
```

Parameters

env	

Returns

Implements ExprNode.

5.19.3.2 eval() [2/2]

```
int ExprIdent::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.20 ExprKnow Class Reference

Node for a knowledge operator.

`#include <expressions.hpp>`

Public Member Functions

- ExprKnow (string _agentName, ExprNode *_arg)
  Constant expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.20.1 Detailed Description

Node for a knowledge operator.

5.20.2 Constructor & Destructor Documentation

5.20.2.1 ExprKnow()

```
ExprKnow::ExprKnow (
            string _agentName,
            ExprNode * _arg )  [inline]
```

Constant expression constructor.

Parameters

_agentName	Agent name for knowledge operator.
_arg	Expression to verify with the given knowledge.

5.20.3 Member Function Documentation

5.20.3.1 eval() [1/2]

```
int ExprKnow::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.20.3.2 eval() [2/2]

```
int ExprKnow::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.21 ExprLe Class Reference

Node for "<=" operator.

`#include <expressions.hpp>`

Public Member Functions

- ExprLe (ExprNode *_larg, ExprNode *_rarg)
  Less or equal expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.21.1 Detailed Description

Node for "<=" operator.

5.21.2 Constructor & Destructor Documentation

5.21.2.1 ExprLe()

```
ExprLe::ExprLe (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Less or equal expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.21.3 Member Function Documentation

5.21.3.1 eval() [1/2]

```
int ExprLe::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.21.3.2 eval() [2/2]

```
int ExprLe::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.22 ExprLt Class Reference

Node for "<" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprLt (ExprNode *_larg, ExprNode *_rarg)

  Less than expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int eval (Environment &env)

5.22.1 Detailed Description

Node for "<" operator.

5.22.2 Constructor & Destructor Documentation

5.22.2.1 ExprLt()

```
ExprLt::ExprLt (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Less than expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.22.3 Member Function Documentation

5.22.3.1 eval() [1/2]

```
int ExprLt::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.22.3.2 eval() [2/2]

```
int ExprLt::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.23 ExprMul Class Reference

Node for multiplication.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprMul (ExprNode *_larg, ExprNode *_rarg)
  Multiplication expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.23.1 Detailed Description

Node for multiplication.

5.23.2 Constructor & Destructor Documentation

5.23.2.1 ExprMul()

```
ExprMul::ExprMul (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Multiplication expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.23.3 Member Function Documentation

5.23.3.1 eval() [1/2]

```
int ExprMul::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.23.3.2 eval() [2/2]

```
int ExprMul::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.24 ExprNe Class Reference

Node for "!=" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprNe (ExprNode *_larg, ExprNode *_rarg)
  Not equals expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.24.1 Detailed Description

Node for "!=" operator.

5.24.2 Constructor & Destructor Documentation

5.24.2.1 ExprNe()

```
ExprNe::ExprNe (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Not equals expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.24.3 Member Function Documentation

5.24.3.1 eval() [1/2]

```
int ExprNe::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.24.3.2 eval() [2/2]

```
int ExprNe::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.25 ExprNode Class Reference

Base node for expressions.

`#include <expressions.hpp>`

Public Member Functions

- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)=0
  Calculates the expression value.
- virtual int eval (Environment &env)=0

5.25.1 Detailed Description

Base node for expressions.

5.25.2 Member Function Documentation

5.25.2.1 eval() [1/2]

```
virtual int ExprNode::eval (
            Environment & env )  [pure virtual]
```

Implemented in ExprIdent.

5.25.2.2 eval() [2/2]

```
virtual int ExprNode::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [pure virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implemented in ExprConst, ExprIdent, ExprAdd, ExprSub, ExprMul, ExprDiv, ExprRem, ExprAnd, ExprOr, ExprNot, ExprEq, ExprNe, ExprLt, ExprLe, ExprGt, ExprGe, ExprKnow, and ExprHart.

The documentation for this class was generated from the following file:

- expressions.hpp

5.26 ExprNot Class Reference

Node for NOT operator.

`#include <expressions.hpp>`

Public Member Functions

- ExprNot (ExprNode *_arg)
  Logic NOT expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.26.1 Detailed Description

Node for NOT operator.

5.26.2 Constructor & Destructor Documentation

5.26.2.1 ExprNot()

```
ExprNot::ExprNot (
            ExprNode * _arg )  [inline]
```

Logic NOT expression constructor.

Parameters

_arg	Calculates the expression value.

5.26.3 Member Function Documentation

5.26.3.1 eval() [1/2]

```
int ExprNot::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.26.3.2 eval() [2/2]

```
int ExprNot::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.27 ExprOr Class Reference

Node for OR operator.

`#include <expressions.hpp>`

Public Member Functions

- ExprOr (ExprNode *_larg, ExprNode *_rarg)
  Logic OR expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.27.1 Detailed Description

Node for OR operator.

5.27.2 Constructor & Destructor Documentation

5.27.2.1 ExprOr()

```
ExprOr::ExprOr (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Logic OR expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.27.3 Member Function Documentation

5.27.3.1 eval() [1/2]

```
int ExprOr::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.27.3.2 eval() [2/2]

```
int ExprOr::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.28 ExprRem Class Reference

Node for modulo.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprRem (ExprNode *_larg, ExprNode *_rarg)

  Modulo expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int eval (Environment &env)

5.28.1 Detailed Description

Node for modulo.

5.28.2 Constructor & Destructor Documentation

5.28.2.1 ExprRem()

```
ExprRem::ExprRem (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Modulo expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.28.3 Member Function Documentation

5.28.3.1 eval() [1/2]

```
int ExprRem::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.28.3.2 eval() [2/2]

```
int ExprRem::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.29 ExprSub Class Reference

Node for subtraction.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprSub (ExprNode *_larg, ExprNode *_rarg)
  Subtraction expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int eval (Environment &env)

5.29.1 Detailed Description

Node for subtraction.

5.29.2 Constructor & Destructor Documentation

5.29.2.1 ExprSub()

```
ExprSub::ExprSub (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Subtraction expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.29.3 Member Function Documentation

5.29.3.1 eval() [1/2]

```
int ExprSub::eval (
            Environment & env )  [virtual]
```

Implements ExprNode.

5.29.3.2 eval() [2/2]

```
int ExprSub::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.30 Formula Struct Reference

Contains the processed verification formula.

`#include <Types.hpp>`

Public Attributes

- set< Agent * > **coalition**

  Coalition of Agent from the formula.
- vector< ExprNode * > * **p**
- bool **isF**
- bool **isCTL**
- float **probability** = 1
- ProbabilitySign **probabilitySign** = ProbabilitySign::NONE

5.30.1 Detailed Description

Contains the processed verification formula.

The documentation for this struct was generated from the following file:

- Types.hpp

5.31 FormulaTemplate Struct Reference

Contains a template for coalition of Agent as string from the formula.

`#include <Types.hpp>`

Public Attributes

- set< string > * **coalition**
- vector< ExprNode * > * **formula**
- bool **isF**
- ProbNode * **probability**
- string **probabilitySign** = ""

5.31.1 Detailed Description

Contains a template for coalition of Agent as string from the formula.

The documentation for this struct was generated from the following file:

- Types.hpp

5.32 GlobalModel Struct Reference

Represents a global model, containing agents and a formula.

`#include <GlobalModel.hpp>`

Public Attributes

- vector< Agent * > **agents**

  Pointers to all agents in a model.
- Formula * **formula**

  A pointer to a Formula.
- GlobalState * **initState**

  Pointer to the initial state of the model.
- vector< GlobalState * > **globalStates**

  Every GlobalState in the model.
- map< Agent *, map< string, EpistemicClass * > > **epistemicClasses**

  Map of Agent pointers to a map of EpistemicClass for graph traversal.
- map< Agent *, map< string, set< GlobalState * > > > **epistemicClassesKnowledge**

  Map of Agent pointers to a map of EpistemicClass for knowledge checks.

5.32.1 Detailed Description

Represents a global model, containing agents and a formula.

The documentation for this struct was generated from the following file:

- GlobalModel.hpp

5.33 GlobalModelGenerator Class Reference

Stores the local models, formula and a global model.

`#include <GlobalModelGenerator.hpp>`

Classes

- struct GlobalStateTupleHash

Public Member Functions

- **GlobalModelGenerator** ()

  Constructor for GlobalModelGenerator class.
- ∼**GlobalModelGenerator** ()

  Destructor for GlobalModelGenerator class.
- GlobalState ∗ initModel (LocalModels ∗localModels, Formula ∗formula)

  Initializes a global model from local models and a formula.
- void expandState (GlobalState ∗state)

  Goes through all GlobalTransition in a given GlobalState and creates new GlobalStates connected to the given one.
- vector< GlobalState ∗ > expandStateAndReturn (GlobalState ∗state, bool returnAnyway=false)

  GlobalModelGenerator::expandState that also additionally returns a vector of newly created states.
- void **expandAllStates** (bool additionalProbSplit=false)

  Expands the states starting from the initial GlobalState and continues until there are no more states to expand.
- void expandAndReduceAllStates ()

  Expands and reduces the states starting from the initial GlobalState using a DFS-POR algorithm and continues until there are no more states to expand.
- GlobalModel ∗ getCurrentGlobalModel ()

  Get for a GlobalModel used in initialization.
- Formula ∗ getFormula ()

  Get for the Formula used in initialization.
- int getFormulaSize ()

  Get size of the Formula used in initialization.
- set< GlobalState ∗ > ∗ findOrCreateEpistemicClassForKnowledge (vector< LocalState ∗ > ∗localStates, GlobalState ∗globalState, Agent ∗agent)

  Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.
- Agent ∗ getAgentInstanceByName (string agentName)

  Get a pointer to an agent by using its name.
- void **markFormulaAsIncorrect** ()

  Sets formula to an incorrectly written state.

- bool getFormulaCorectness ()

  Gets formula status.
- void initStrategy (StrategyCollection *strat)

  Initiates the strategy with a given StrategyCollection.
- set< set< tuple< string, string > > > createProbabilityStrategy (LocalModels *localModels)

  Prepares the strategy generating structures for the probabilistic verification.
- set< tuple< string, string > > * getNextPath ()

  Retrieves the next strategy one at a time (iteratively). Systematically tries all combinations of action choices at each epistemic class.
- MDP **generateNextMDP** (bool makeOpponentGoMax=false)
- string getCoalitionIdentifier (vector< LocalState * > *localStates)

  Returns a concatenated epistemic class ID of all coalition members in given LocalStates.
- string **getCoalitionActionSignature** (GlobalTransition *transition, char sep=';')

  Returns coalition-only action signature, ordered by agentIndex and using localName when available.
- string getActionNameFromStateInStrategy (GlobalState *state)

  Gives next action's name from a given state.

Public Attributes

- map< Agent *, size_t > **agentIndex**

  auxiliary variable mapping Agent pointer to its index (replace size_t with if needed later)

Protected Member Functions

- GlobalState * generateInitState ()

  Generates initial state of the model from GlobalModel in memory.
- GlobalState * generateStateFromLocalStates (vector< LocalState * > *localStates, set< LocalTransition * > *viaLocalTransitions, GlobalState *prevGlobalState)

  Creates a new GlobalState using some of the internally known model data and given local states, transitions that were uset to get there and the previous global state.
- void generateGlobalTransitions (GlobalState *fromGlobalState, set< LocalTransition * > localTransitions, map< Agent *, vector< LocalTransition * > > transitionsByAgent)

  Adds all shared global transitions to a GlobalState.
- string computeEpistemicClassHash (vector< LocalState * > *localStates, Agent *agent)

  Creates a hash from a set of LocalState and an Agent.
- string computeGlobalStateHash (vector< LocalState * > *localStates)

  Creates a hash from a set of LocalState.
- EpistemicClass * findOrCreateEpistemicClass (vector< LocalState * > *localStates, Agent *agent)

  Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.
- GlobalState * findGlobalStateInEpistemicClass (vector< LocalState * > *localStates, EpistemicClass *epistemicClass)

  Gets a GlobalState from an EpistemicClass if it exists in that episcemic class.
- bool **checkLocalStates** (vector< LocalState * > *localStates, GlobalState *globalState)

Protected Attributes

- LocalModels * **localModels**

  LocalModels used in initModel.
- Formula * **formula**

  Formula used in initModel.
- GlobalModel * **globalModel**

  GlobalModel created in initModel.
- bool **correctModel**

  Flag holding info if model is actually correct.
- unordered_map< GlobalState *, unordered_set< int > > **candidateStateDepths**

  Lookup map containing global states and the depths that the global state is contained in. Used for reductions.
- stack< GlobalState * > **globalModelCandidates**

  Candidate states to be used in reductions. Used for reductions.
- stack< int > **stateDepths**

  Saved depths of states added to statesToExpand. Used for reductions.
- unordered_set< GlobalState * > **addedStates**

  States that were added to a stack of states. Used for reductions.
- StrategyCollection * **strategyCollection**

  Strategy to check during verification (if any)
- set< tuple< string, string > > **currentStrategy**
- map< string, size_t > **choiceIndices**
- map< string, size_t > **actionCounts**
- bool **strategyGenerationInit** = false
- bool **strategiesExhausted** = false
- unordered_map< string, GlobalState * > **stateHashMapCache**
- map< string, map< string, set< GlobalTransition * > > > **coalitionTransitions**
- map< string, map< string, set< GlobalTransition * > > > **opponentsTransitions**

5.33.1 Detailed Description

Stores the local models, formula and a global model.

5.33.2 Member Function Documentation

5.33.2.1 computeEpistemicClassHash()

```
string GlobalModelGenerator::computeEpistemicClassHash (
            vector< LocalState * > * localStates,
            Agent * agent )  [protected]
```

Creates a hash from a set of LocalState and an Agent.

Parameters

localStates	Pointer to a vector of pointers of LocalState and pointer to and Agent to turn into a hash.

Returns

Returns a string with a hash.

5.33.2.2 computeGlobalStateHash()

```
string GlobalModelGenerator::computeGlobalStateHash (
            vector< LocalState * > * localStates )  [protected]
```

Creates a hash from a set of LocalState.

Parameters

localStates	Pointer to a vector of pointers of LocalState to turn into a hash.

Returns

Returns a string with a hash.

5.33.2.3 createProbabilityStrategy()

```
set< set< tuple< string, string > > > GlobalModelGenerator::createProbabilityStrategy (
            LocalModels * localModels )
```

Prepares the strategy generating structures for the probabilistic verification.

Parameters

localModels	LocalModels to generate the coalition transition buckets from.

5.33.2.4 expandAndReduceAllStates()

```
void GlobalModelGenerator::expandAndReduceAllStates ( )
```

Expands and reduces the states starting from the initial GlobalState using a DFS-POR algorithm and continues until there are no more states to expand.

Parameters

depth	Current depth of the recursive generation of all states.

5.33.2.5 expandState()

```
void GlobalModelGenerator::expandState (
            GlobalState * state )
```

Goes through all GlobalTransition in a given GlobalState and creates new GlobalStates connected to the given one.

Parameters

state	A state from which the expansion should start.

5.33.2.6 expandStateAndReturn()

```
vector< GlobalState * > GlobalModelGenerator::expandStateAndReturn (
            GlobalState * state,
            bool returnAnyway = false )
```

GlobalModelGenerator::expandState that also additionally returns a vector of newly created states.

Parameters

state	A state from which the expansion should start.

5.33.2.7 findGlobalStateInEpistemicClass()

```
GlobalState * GlobalModelGenerator::findGlobalStateInEpistemicClass (
            vector< LocalState * > * localStates,
            EpistemicClass * epistemicClass )  [protected]
```

Gets a GlobalState from an EpistemicClass if it exists in that episcemic class.

Parameters

localStates	Pointer to a vector of pointers to LocalState, from which will be generated a global state hash.
epistemicClass	Epistemic class in which to check if a GlobalState exists.

Returns

Returns a pointer to a GlobalState if it exists in given epistemic class, otherwise returns nullptr.

5.33.2.8 findOrCreateEpistemicClass()

```
EpistemicClass * GlobalModelGenerator::findOrCreateEpistemicClass (
            vector< LocalState * > * localStates,
            Agent * agent )  [protected]
```

Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.

Parameters

localStates	Local states from agent.
agent	Agent for which to check the existence of an epistemic class.

Returns

A pointer to a new or existing EpistemicClass.

5.33.2.9 findOrCreateEpistemicClassForKnowledge()

```
set< GlobalState * > * GlobalModelGenerator::findOrCreateEpistemicClassForKnowledge (
            vector< LocalState * > * localStates,
            GlobalState * global,
            Agent * agent )
```

Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.

Parameters

localStates	Local states from agent.
agent	Agent for which to check the existence of an epistemic class.

Returns

A pointer to a new or existing EpistemicClass.

5.33.2.10 generateGlobalTransitions()

```
void GlobalModelGenerator::generateGlobalTransitions (
            GlobalState * fromGlobalState,
            set< LocalTransition * > localTransitions,
            map< Agent *, vector< LocalTransition * > > transitionsByAgent )  [protected]
```

Adds all shared global transitions to a GlobalState.

Parameters

fromGlobalState	Global state to add transitions to.
localTransitions	Initially empty, avaliable local transitions by each agent from transitionsByAgent.
transitionsByAgent	Mapped transitions to an agent, only with transitions avaliable for the agent at this moment.

5.33.2.11 generateInitState()

```
GlobalState * GlobalModelGenerator::generateInitState ( )  [protected]
```

Generates initial state of the model from GlobalModel in memory.

Returns

Returns a pointer to an initial GlobalState.

5.33.2.12 generateStateFromLocalStates()

```
GlobalState * GlobalModelGenerator::generateStateFromLocalStates (
            vector< LocalState * > * localStates,
            set< LocalTransition * > * viaLocalTransitions,
            GlobalState * prevGlobalState )  [protected]
```

Creates a new GlobalState using some of the internally known model data and given local states, transitions that were uset to get there and the previous global state.

Parameters

localStates	LocalStates from which the new GlobalState will be built.
viaLocalTransitions	Pointer to a set of pointers to LocalTransition from which the changes in variables, as a result of traversing through the transition, will be made in a new GlobalState.
prevGlobalState	Pointer to GlobalState from which all persistent variables will be copied over from to the new GlobalState.

Returns

Returns a pointer to a new or already existing in the same epistemic class GlobalModel.

5.33.2.13 getActionNameFromStateInStrategy()

```
string GlobalModelGenerator::getActionNameFromStateInStrategy (
            GlobalState * state )
```

Gives next action's name from a given state.

Parameters

state	GlobalState from which an action name would be retrieved if a set strategy is present.

Returns

String containing the action name ending on a semicolon.

5.33.2.14 getAgentInstanceByName()

```
Agent * GlobalModelGenerator::getAgentInstanceByName (
            string agentName )
```

Get a pointer to an agent by using its name.

Parameters

agentName	Name of an Agent that we want to get its instance.

Returns

Pointer to an Agent.

5.33.2.15 getCoalitionIdentifier()

```
string GlobalModelGenerator::getCoalitionIdentifier (
            vector< LocalState * > * localStates )
```

Returns a concatenated epistemic class ID of all coalition members in given LocalStates.

Parameters

localStates	Vector of LocalStates to extract coalition IDs from.

Returns

LocalState IDs for coalition members separated with semicolons, ending on a semicolon.

5.33.2.16 getCurrentGlobalModel()

```
GlobalModel * GlobalModelGenerator::getCurrentGlobalModel ( )
```

Get for a GlobalModel used in initialization.

Returns

Returns a pointer to a global model.

5.33.2.17 getFormula()

```
Formula * GlobalModelGenerator::getFormula ( )
```

Get for the Formula used in initialization.

Returns

Returns a pointer to the formula structure.

5.33.2.18 getFormulaCorectness()

```
bool GlobalModelGenerator::getFormulaCorectness ( )
```

Gets formula status.

Returns

True if formula is written correctly, false otherwise.

5.33.2.19 getFormulaSize()

```
int GlobalModelGenerator::getFormulaSize ( )
```

Get size of the Formula used in initialization.

Returns

Returns the formula size.

5.33.2.20 getNextPath()

```
set< tuple< string, string > > * GlobalModelGenerator::getNextPath ( )
```

Retrieves the next strategy one at a time (iteratively). Systematically tries all combinations of action choices at each epistemic class.

Returns

Pointer to next strategy, or nullptr if all exhausted.

5.33.2.21 initModel()

```
GlobalState * GlobalModelGenerator::initModel (
            LocalModels * localModels,
            Formula * formula )
```

Initializes a global model from local models and a formula.

Parameters

localModels	Pointer to LocalModels that will construct a global model.
formula	Pointer to a Formula to include into the model.

Returns

Returns a pointer to initial state of the global model.

5.33.2.22 initStrategy()

```
void GlobalModelGenerator::initStrategy (
            StrategyCollection * strat )
```

Initiates the strategy with a given StrategyCollection.

Parameters

strat	StrategyCollection to initialize the strategy.

The documentation for this class was generated from the following files:

- GlobalModelGenerator.hpp
- GlobalModelGenerator.cpp

5.34 GlobalState Struct Reference

Represents a single global state.

`#include <GlobalState.hpp>`

Public Member Functions

- std::string toString (string indent="")

  Debug information on the given GlobalState.
- map< string, int > getGlobalStateEnvironment ()

  Get for the environment of a given global state.

Public Attributes

- string **hash**

  Hash of the global state used in quick checks if the states are in the same epistemic class.
- map< Agent *, EpistemicClass * > **epistemicClasses**

  Map of agents and the epistemic classes that belongs to the respective agent.
- bool **isExpanded**

  If false, the state can be still expanded, potentially creating new states, otherwise the expansion of the state already occured and is not necessary.
- GlobalStateVerificationStatus **verificationStatus** = GLOBAL_STATE_VERIFICATION_STATUS::UNVERIFIED

  Current verifivation status of this state.
- float **probability** = config.probability ? 0.0 : 1.0

  Collective probability of the state.
- bool **goodState** = false
- set< GlobalTransition * > **globalTransitions**

  Every GlobalTransition in the model.
- set< GlobalState * > **preimage**

  Preimage for the global state, i.e. states from which there is a transition to this state.
- vector< LocalState * > **localStatesProjection**

  Local states of each agent that define this global state.
- map< Agent *, set< GlobalState * > * > **epistemicClassesAllAgents**

  States in the same epistemic class as the current one, for KBC.
- ProbabilityEntry **probabilityResult**
- GlobalTransition * **probabilityLoop** = nullptr

  Used to resolve self loops during probability calculation.
- VerifResult **stateVerifResult** = VerifResult::NOT_VERIFIED

5.34.1 Detailed Description

Represents a single global state.

5.34.2 Member Function Documentation

5.34.2.1 getGlobalStateEnvironment()

```
map< string, int > GlobalState::getGlobalStateEnvironment ( )
```

Get for the environment of a given global state.

Returns

A map of variable names and their values for the current global state.

5.34.2.2 toString()

```
std::string GlobalState::toString (
            string indent = "" )
```

Debug information on the given GlobalState.

Parameters

indent	- optional indentation string

Returns

GlobalState data

GlobalState data

The documentation for this struct was generated from the following files:

- GlobalState.hpp
- GlobalState.cpp

5.35 GlobalModelGenerator::GlobalStateTupleHash Struct Reference

Public Member Functions

- size_t **operator()** (const tuple< GlobalState *, int > &t) const

The documentation for this struct was generated from the following file:

- GlobalModelGenerator.hpp

5.36 GlobalTransition Struct Reference

Represents a single global transition.

```
#include <GlobalTransition.hpp>
```

Public Member Functions

- **GlobalTransition** ()

  Constructor for GlobalTransition.
- string joinLocalTransitionNames (char sep=';')

  Concatenates local transition names into a string.
- float **getProbability** ()

Public Attributes

- uint32_t **id**

  Identifier of the transition.
- bool **isInvalidDecision**

  Marks if the transition is invalid, true if there is no point in traversing that transition, otherwise false.
- GlobalState * **from**

  Binding to a GlobalState from which this transition goes from.
- GlobalState * **to**

  Binding to a GlobalState from which this transition goes to.
- set< LocalTransition * > **localTransitions**

  Local transitions that define this global transition. A single transition or more in case of shared transitions.

Static Public Attributes

- static atomic_uint32_t **next_id**

5.36.1 Detailed Description

Represents a single global transition.

5.36.2 Member Function Documentation

5.36.2.1 joinLocalTransitionNames()

```
string GlobalTransition::joinLocalTransitionNames (
            char sep = ';' )
```

Concatenates local transition names into a string.

Parameters

sep	Separator used for separating the transition names.

Returns

String of transition names joined with the given separator and ending on the separator.

The documentation for this struct was generated from the following files:

- GlobalTransition.hpp
- GlobalTransition.cpp

5.37 HistoryDbg Class Reference

Stores history and allows displaying it to the console.

`#include <Verification.hpp>`

Public Member Functions

- **HistoryDbg** ()
  A constructor for HistoryDbg.
- ~**HistoryDbg** ()
  A destructor for HistoryDbg.
- void addEntry (HistoryEntry *entry)
  Adds a HistoryEntry to the debug history.
- void markEntry (HistoryEntry *entry, char chr)
  Marks an entry in the degug history with a char.
- void print (string prefix)
  Prints every entry from the algorithm's path.
- HistoryEntry * cloneEntry (HistoryEntry *entry)
  Checks if the HistoryEntry pointer exists in the debug history.

Public Attributes

- vector< pair< HistoryEntry *, char > > **entries**
  A pair of history entries and a char marking history type.

5.37.1 Detailed Description

Stores history and allows displaying it to the console.

5.37.2 Member Function Documentation

5.37.2.1 addEntry()

```
void HistoryDbg::addEntry (
            HistoryEntry * entry )
```

Adds a HistoryEntry to the debug history.

Parameters

entry	A pointer to the HistoryEntry that will be added to the history.

5.37.2.2 cloneEntry()

```
HistoryEntry * HistoryDbg::cloneEntry (
            HistoryEntry * entry )
```

Checks if the HistoryEntry pointer exists in the debug history.

Parameters

entry	A pointer to a HistoryEntry to be checked.

Returns

Identity function if the entry is in history, otherwise returns nullptr.

5.37.2.3 markEntry()

```
void HistoryDbg::markEntry (
            HistoryEntry * entry,
            char chr )
```

Marks an entry in the degug history with a char.

Parameters

entry	A pointer to a HistoryEntry that is supposed to be marked.
chr	A char that will be made into a pair with a HistoryEntry.

5.37.2.4 print()

```
void HistoryDbg::print (
            string prefix )
```

Prints every entry from the algorithm's path.

Parameters

prefix	A prefix string to append to the front of every entry.

The documentation for this class was generated from the following files:

- Verification.hpp
- Verification.cpp

5.38 HistoryEntry Struct Reference

Structure used to save model traversal history.

`#include <Verification.hpp>`

Public Member Functions

- string toString ()

  Converts HistoryEntry to string.

Public Attributes

- HistoryEntryType **type**

  Type of the history record.
- GlobalState * **globalState**

  Saved global state.
- GlobalTransition * **decision**

  Selected transition.
- bool **globalTransitionControlled**

  Is the transition controlled by an agent in coalition.
- GlobalStateVerificationStatus **prevStatus**

  Previous model verification state.
- GlobalStateVerificationStatus **newStatus**

  Next model verification state.
- int **depth**

  Recursion depth.
- StrategyEntry **strategy**

  Holds currently processed strategy for the current state.
- HistoryEntry * **prev**

  Pointer to the previous HistoryEntry.
- HistoryEntry * **next**

  Pointer to the next HistoryEntry.

5.38.1 Detailed Description

Structure used to save model traversal history.

5.38.2 Member Function Documentation

5.38.2.1 toString()

```
string HistoryEntry::toString ( )  [inline]
```

Converts HistoryEntry to string.

Returns

A string with the descriprion of this history record.

The documentation for this struct was generated from the following file:

- Verification.hpp

5.39 LocalModels Struct Reference

Represents a single local model, contains all agents and variables.

`#include <Types.hpp>`

Public Attributes

- vector< Agent * > **agents**
  A vector of agents for the current model.

5.39.1 Detailed Description

Represents a single local model, contains all agents and variables.

The documentation for this struct was generated from the following file:

- Types.hpp

5.40 LocalState Class Reference

Represents a single LocalState, containing id, name and internal variables.

`#include <LocalState.hpp>`

Public Member Functions

- bool compare (LocalState *state)
  Function comparing two states.
- string toString (string indent="")
  Debug information on the given LocalState.

Public Attributes

- uint32_t **id**
  State identifier.
- string **name**
  State name.
- map< string, int > **environment**
  Local variables as a name and their current values.
- Agent * **agent**
  Binding to an Agent.
- set< LocalTransition * > **localTransitions**
  Binding to the set of LocalTransition.
- set< GlobalState * > **epistemicGlobalStates**
  Binding to the set of GlobalState that this LocalState belongs to.

5.40.1 Detailed Description

Represents a single LocalState, containing id, name and internal variables.

5.40.2 Member Function Documentation

5.40.2.1 compare()

```
bool LocalState::compare (
            LocalState * state )
```

Function comparing two states.

Parameters

state	A pointer to LocalState to which this state should be compared to.

Returns

Returns true if the current LocalState is the same as the passed one, otherwise false.

5.40.2.2 toString()

```
string LocalState::toString (
            string indent = "" )
```

Debug information on the given LocalState.

Parameters

indent	- optional indentation string

Returns

LocalState data

LocalState data

The documentation for this class was generated from the following files:

- LocalState.hpp
- LocalState.cpp

5.41 LocalStateTemplate Class Reference

A template for the local state.

```
#include <nodes.hpp>
```

Public Attributes

- string **name**

  Name of the local state.
- set< TransitionTemplate * > **transitions**

  Local transitions going out from this state.

5.41.1 Detailed Description

A template for the local state.

The documentation for this class was generated from the following file:

- nodes.hpp

5.42 LocalTransition Struct Reference

Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

`#include <LocalTransition.hpp>`

Public Attributes

- int **id**

  Identifier of the transition.
- string **name**

  Name of the transition (global).
- string **localName**

  Name of the transition (local).
- bool **isShared**

  Is the transition appearing somewhere else, true if yes, false if no.
- int **sharedCount**

  Count of recurring appearances of this transition.
- set< Condition * > **conditions**

  Conditions that have to be fulfilled for the transition to be avaliable.
- float **probability**

  Used for probability formula verification. The probability of this LocalTransition executing.
- Agent * **agent**

  Binding to an Agent.
- LocalState * **from**

  Binding to a LocalState from which this transition goes from.
- LocalState * **to**

  Binding to a LocalState from which this transition goes to.

5.42.1 Detailed Description

Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

The documentation for this struct was generated from the following file:

- LocalTransition.hpp

5.43 ModelParser Class Reference

A parser for converting a text file into a model.

```
#include <ModelParser.hpp>
```

Public Member Functions

- **ModelParser** ()
  ModelParser constructor.
- ~**ModelParser** ()
  ModelParser destructor.
- tuple< LocalModels, Formula > parse (string fileName)
  Parses a file with given name into a usable model.
- tuple< LocalModels, Formula > parseAndOverwriteFormula (string fileName, string s)
  Parses a text file in the given file path and internally replaces the verification formula with the given one.

5.43.1 Detailed Description

A parser for converting a text file into a model.

5.43.2 Member Function Documentation

5.43.2.1 parse()

```
tuple< LocalModels, Formula > ModelParser::parse (
            string fileName )
```

Parses a file with given name into a usable model.

Parameters

fileName	Name of the file to be converted into a model.

Returns

Pointer to a model created from a given file.

5.43.2.2 parseAndOverwriteFormula()

```
tuple< LocalModels, Formula > ModelParser::parseAndOverwriteFormula (
            string fileName,
            string s )
```

Parses a text file in the given file path and internally replaces the verification formula with the given one.

Parameters

fileName	Path to the text file.
s	New formula.

Returns

Returns a new model generation structure with a replaced formula.

The documentation for this class was generated from the following files:

- ModelParser.hpp
- ModelParser.cc

5.44 ProbabilityEntry Class Reference

Class for handling the probability operations during probability verification.

```
#include <TypesDependency.hpp>
```

Public Member Functions

- **ProbabilityEntry** (ProbabilityCalculationType mode)
- void **changeVerificationMode** (ProbabilityCalculationType mode)
- ProbabilityCalculationType **getVerificationMode** ()
- void **changeSumProbabilityTrue** (float changeProbabilityBy)
- void **changeSumProbabilityFalse** (float changeProbabilityBy)
- ProbabilityTrueFalse **getSumProbability** ()
- void **setSumProbability** (ProbabilityTrueFalse newSumProbability)
- void **changeMinProbabilityTrue** (float newProbability)
- void **changeMinProbabilityFalse** (float newProbability)
- ProbabilityTrueFalse **getMinProbability** ()
- void **setMinProbability** (ProbabilityTrueFalse newMinProbability)
- ProbabilityTrueFalse **returnProbability** ()

5.44.1 Detailed Description

Class for handling the probability operations during probability verification.

The documentation for this class was generated from the following file:

- TypesDependency.hpp

5.45 ProbabilityTrueFalse Struct Reference

Contains probability of a state being in a true or false state.

`#include <TypesDependency.hpp>`

Public Attributes

- float **probabilityTrue** = 0.0
- float **probabilityFalse** = 0.0

5.45.1 Detailed Description

Contains probability of a state being in a true or false state.

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.46 ProbAdd Class Reference

Node for addition.

`#include <expressions.hpp>`

Public Member Functions

- ProbAdd (ProbNode *_larg, ProbNode *_rarg)
  Addition expression constructor.
- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual float eval ()

5.46.1 Detailed Description

Node for addition.

5.46.2 Constructor & Destructor Documentation

5.46.2.1 ProbAdd()

```
ProbAdd::ProbAdd (
            ProbNode * _larg,
            ProbNode * _rarg )  [inline]
```

Addition expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.46.3 Member Function Documentation

5.46.3.1 eval() [1/2]

```
float ProbAdd::eval ( )  [virtual]
```

Implements ProbNode.

5.46.3.2 eval() [2/2]

```
float ProbAdd::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.47 ProbConst Class Reference

Node for a constant.

```
#include <expressions.hpp>
```

Public Member Functions

- ProbConst (float _val)

  Constant expression constructor.
- virtual float eval (GlobalModelGenerator ∗generator, GlobalState ∗globalState)

  Calculates the expression value.
- virtual float eval ()

5.47.1 Detailed Description

Node for a constant.

5.47.2 Constructor & Destructor Documentation

5.47.2.1 ProbConst()

```
ProbConst::ProbConst (
            float _val )  [inline]
```

Constant expression constructor.

Parameters

_val	ProbConst value.

5.47.3 Member Function Documentation

5.47.3.1 eval() [1/2]

```
float ProbConst::eval ( )  [virtual]
```

Implements ProbNode.

5.47.3.2 eval() [2/2]

```
float ProbConst::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.48 ProbDiv Class Reference

Node for division.

`#include <expressions.hpp>`

Public Member Functions

- ProbDiv (ProbNode ∗_larg, ProbNode ∗_rarg)
  Division expression constructor.
- virtual float eval (GlobalModelGenerator ∗generator, GlobalState ∗globalState)
  Calculates the expression value.
- virtual float eval ()

5.48.1 Detailed Description

Node for division.

5.48.2 Constructor & Destructor Documentation

5.48.2.1 ProbDiv()

```
ProbDiv::ProbDiv (
            ProbNode * _larg,
            ProbNode * _rarg )  [inline]
```

Division expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.48.3 Member Function Documentation

5.48.3.1 eval() [1/2]

```
float ProbDiv::eval ( )  [virtual]
```

Implements ProbNode.

5.48.3.2 eval() [2/2]

```
float ProbDiv::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.49 ProbMul Class Reference

Node for multiplication.

```
#include <expressions.hpp>
```

Public Member Functions

- ProbMul (ProbNode *_larg, ProbNode *_rarg)
  Multiplication expression constructor.
- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual float eval ()

5.49.1 Detailed Description

Node for multiplication.

5.49.2 Constructor & Destructor Documentation

5.49.2.1 ProbMul()

```
ProbMul::ProbMul (
            ProbNode * _larg,
            ProbNode * _rarg )  [inline]
```

Multiplication expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.49.3 Member Function Documentation

5.49.3.1 eval() [1/2]

```
float ProbMul::eval ( )  [virtual]
```

Implements ProbNode.

5.49.3.2 eval() [2/2]

```
float ProbMul::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.50 ProbNode Class Reference

Base node for probability calculations.

```
#include <expressions.hpp>
```

Public Member Functions

- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)=0
  Calculates the expression value.
- virtual float **eval** ()=0

5.50.1 Detailed Description

Base node for probability calculations.

5.50.2 Member Function Documentation

5.50.2.1 eval()

```
virtual float ProbNode::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [pure virtual]
```

Calculates the expression value.

Returns

Returns a float.

Implemented in ProbConst, ProbAdd, ProbSub, ProbMul, and ProbDiv.

The documentation for this class was generated from the following file:

- expressions.hpp

5.51 ProbSub Class Reference

Node for subtraction.

```
#include <expressions.hpp>
```

Public Member Functions

- ProbSub (ProbNode *_larg, ProbNode *_rarg)
  Subtraction expression constructor.
- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual float eval ()

5.51.1 Detailed Description

Node for subtraction.

5.51.2 Constructor & Destructor Documentation

5.51.2.1 ProbSub()

```
ProbSub::ProbSub (
            ProbNode * _larg,
            ProbNode * _rarg )  [inline]
```

Subtraction expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.51.3 Member Function Documentation

5.51.3.1 eval() [1/2]

```
float ProbSub::eval ( )  [virtual]
```

Implements ProbNode.

5.51.3.2 eval() [2/2]

```
float ProbSub::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.52 Result Struct Reference

Public Attributes

- bool **verificationResult** = false
- ProbabilityTrueFalse **probabilityResult**

The documentation for this struct was generated from the following file:

- Types.hpp

5.53 StateVerificationInfo Struct Reference

Handles all single state verification information during probabilistic verification.

`#include <TypesDependency.hpp>`

Public Attributes

- GlobalState * **globalState** = nullptr
- StateVerificationInfo * **fromState** = nullptr
- queue< GlobalTransition * > **controlledTransitionsLeftToProcess**
- queue< GlobalTransition * > **uncontrolledTransitionsLeftToProcess**
- map< string, set< GlobalTransition * > > **controlledProbabilisticTransitionsLeftToProcess**
- map< string, set< GlobalTransition * > > **uncontrolledProbabilisticTransitionsLeftToProcess**
- int **depth** = 0
- bool **processed** = false
- VerifResult **verifResult** = VerifResult::NOT_VERIFIED
- bool **controlled** = false
- bool **uncontrolled** = false
- bool **hasControlledProbabilistic** = false
- bool **hasUncontrolledProbabilistic** = false
- bool **hasValidControlledTransition** = false
- bool **hasValidUncontrolledTransition** = true
- bool **isControlledByCoalition** = false
- bool **gotThroughPreselectedTransition** = false
- bool **gotResponseFromOtherState** = false

5.53.1 Detailed Description

Handles all single state verification information during probabilistic verification.

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.54 StrategyBitsComparator Struct Reference

A comparator for two bitsets containing values for the global states.

`#include <TypesDependency.hpp>`

Public Member Functions

- bool **operator()** (const bitset< STRATEGY_BITS > &b1, const bitset< STRATEGY_BITS > &b2) const

5.54.1 Detailed Description

A comparator for two bitsets containing values for the global states.

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.55 StrategyCollection Class Reference

Handles currently selected strategy when the strategy is set.

`#include <TypesDependency.hpp>`

Public Member Functions

- **StrategyCollection** (string hash, Action action)
- void **addAction** (Action action)
- void **addStrategy** (StrategyCollection strategy)
- void **clear** ()
- map< string, Action > **getStrategy** ()

5.55.1 Detailed Description

Handles currently selected strategy when the strategy is set.

The documentation for this class was generated from the following file:

- TypesDependency.hpp

5.56 StrategyCollectionTemplate Class Reference

Public Member Functions

- void addAction (vector< string > *_states, string _actionName)

Public Attributes

- map< string, ActionTemplate > **selectedStrategy**

5.56.1 Member Function Documentation

5.56.1.1 addAction()

```
void StrategyCollectionTemplate::addAction (
            vector< string > * _states,
            string _actionName )
```

Parameters

newAction	

The documentation for this class was generated from the following files:

- strategyNodes.hpp
- strategyNodes.cc

5.57 StrategyEntry Struct Reference

Public Attributes

- StrategyEntryType **type** = NOT_MODIFIED
- bitset< STRATEGY_BITS > **globalValues** = 0
- string * **actionName**

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.58 StrategyParser Class Reference

A parser for converting a text file into a strategy.

`#include <StrategyParser.hpp>`

Public Member Functions

- **StrategyParser** ()
  StrategyParser constructor.
- ~**StrategyParser** ()
  ModelParser destructor.
- StrategyCollection * parse (string fileName)
  Parses a file with given name into a usable strategy.

5.58.1 Detailed Description

A parser for converting a text file into a strategy.

5.58.2 Member Function Documentation

5.58.2.1 parse()

```
StrategyCollection * StrategyParser::parse (
            string fileName )
```

Parses a file with given name into a usable strategy.

Parameters

fileName	Name of the file to be converted into a strategy.

Returns

Pointer to a model created from a given file.

The documentation for this class was generated from the following files:

- StrategyParser.hpp
- StrategyParser.cc

5.59 STRATSTYPE Union Reference

Public Attributes

- char * **expr**
- int **val**
- float **floatno**
- char * **ident**
- vector< string > * **identList**

The documentation for this union was generated from the following file:

- strategyParser.h

5.60 TransitionTemplate Class Reference

Represents a meta-transition.

`#include <nodes.hpp>`

Public Member Functions

- TransitionTemplate (int _shared, string _patternName, string _matchName, string _startState, string _endState, ExprNode *_cond, set< Assignment * > *_assign, ProbNode *_prob)

  TransitionTemplate constructor.

Public Attributes

- int **shared**

  Needed amound of needed agents. -1 if not shared.
- string **patternName**

  Name of the pattern.
- string **matchName**

  Global name for shared transitions.
- string **startState**

  Start state name.
- string **endState**

  End state name.
- ExprNode * **condition**

  Condition expression that has do be fulfilled in that transition.
- set< Assignment * > * **assignments**

  Set of assignments.
- ProbNode * **probability**

  Probability of executing the action.

5.60.1 Detailed Description

Represents a meta-transition.

5.60.2 Constructor & Destructor Documentation

5.60.2.1 TransitionTemplate()

```
TransitionTemplate::TransitionTemplate (
            int _shared,
            string _patternName,
            string _matchName,
            string _startState,
            string _endState,
            ExprNode * _cond,
            set< Assignment * > * _assign,
            ProbNode * _prob )  [inline]
```

TransitionTemplate constructor.

Parameters

_shared	Needed amound of needed agents. -1 if not shared.
_patternName	Name of the pattern.
_matchName	Global name for shared transitions.
_startState	Start state name.
_endState	End state name.
_cond	Condition expression that has do be fulfilled in that transition.
_assign	Set of assignments.
_prob	Probability of executing the action.

The documentation for this class was generated from the following file:

- nodes.hpp

5.61 Var Struct Reference

Represents a variable in the model, containing name, initial value and persistence.

`#include <Types.hpp>`

Public Attributes

- string **name**
  Variable name.
- int **initialValue**
  Initial value of the variable.
- bool **persistent**
  True if variable is persistent, i.e. it should appear in all states in the model, false otherwise.
- Agent * **agent**
  Reference to an agent, to which this variable belongs to.

5.61.1 Detailed Description

Represents a variable in the model, containing name, initial value and persistence.

The documentation for this struct was generated from the following file:

- Types.hpp

5.62 Verification Class Reference

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

`#include <Verification.hpp>`

Public Member Functions

- Verification (GlobalModelGenerator *generator)
  Constructor for Verification.
- ~**Verification** ()
  Destructor for Verification.
- bool verify ()
  Starts the process of formula verification on a model.
- bool **fixpointVerify** ()
- void **historyDecisionsERR** ()
  Prints out the first ERR path.
- map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > getNaturalStrategy ()
  Get natural strategy map.
- vector< tuple< vector< tuple< bool, string > >, string > > getReducedStrategy ()
  Takes natural strategy from memory and reduces it using reduceStrategy().
- int getStrategyComplexity ()
  Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.
- Result verifyStrategy ()
  Verifies a strategy with the previously given models and formula.
- Result **verifyMDP** ()

Protected Member Functions

- bool verifyLocalStates (vector< LocalState * > *localStates, GlobalState *globalState)

  Verifies a set of LocalState that a GlobalState is composed of with a hardcoded formula.
- bool verifyGlobalState (GlobalState *globalState, int depth)

  Recursively verifies GlobalState.
- bool isGlobalTransitionControlledByCoalition (GlobalTransition *globalTransition)

  Checks if any of the LocalTransition in a given GlobalTransition has an Agent in a coalition in the formula.
- bool isAgentInCoalition (Agent *agent)

  Checks if the Agent is in a coalition based on the formula in a GlobalModelGenerator.
- EpistemicClass * getEpistemicClassForGlobalState (GlobalState *globalState)

  Gets the EpistemicClass for the agent in passed GlobalState, i.e. transitions from indistinguishable state from certain other states for an agent to other states.
- bool areGlobalStatesInTheSameEpistemicClass (GlobalState *globalState1, GlobalState *globalState2)

  Compares two GlobalState and checks if their EpistemicClass is the same.
- void addHistoryDecision (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type DECISION and puts it on top of the stack of the decision history.
- void addHistoryStateStatus (GlobalState *globalState, GlobalStateVerificationStatus prevStatus, GlobalStateVerificationStatus newStatus)

  Creates a HistoryEntry of the type STATE_STATUS and puts it to the top of the decision history.
- bool addHistoryContext (GlobalState *globalState, int depth, GlobalTransition *decision, bool globalTransitionControlled)

  Creates a HistoryEntry of the type CONTEXT and puts it to the top of the decision history.
- void addHistoryMarkDecisionAsInvalid (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.
- void addHistoryUncontrolledDecision (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type UNCONTROLLED_DECISION and puts it on top of the stack of the decision history.
- HistoryEntry * newHistoryMarkDecisionAsInvalid (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and returns it.
- bool revertLastDecision (int depth)

  Reverts GlobalState and history to the previous decision state.
- void undoLastHistoryEntry (bool freeMemory)

  Removes the top entry of the history stack.
- void undoHistoryUntil (HistoryEntry *historyEntry, bool inclusive, int depth)

  Rolls back the history entries up to the certain HistoryEntry.
- void printCurrentHistory (int depth)

  Prints current history to the console.
- bool equivalentGlobalTransitions (GlobalTransition *globalTransition1, GlobalTransition *globalTransition2)

  Checks if two global transitions are made up of the same local transitions.
- bool checkUncontrolledSet (set< GlobalTransition * > uncontrolledGlobalTransitions, GlobalState *globalState, int depth, bool hasOmittedTransitions, bool mixed=false)

  Verifies if each transition from a given state yields a correct result.
- bool verifyTransitionSets (set< GlobalTransition * > controlledGlobalTransitions, set< GlobalTransition * > uncontrolledGlobalTransitions, GlobalState *globalState, int depth, bool hasOmittedTransitions, bool isFMode, bool mixed=false)

  Checks if given transition sets are able to fulfill the formula for its given epistemic class.
- bool restoreHistory (GlobalState *globalState, GlobalTransition *globalTransition, int depth, bool controlled)

  Restores the decisions made for a given global state and transition in current recursion depth.
- bool **minFixpointVerify** ()
- bool **maxFixpointVerify** ()
- bitset< STRATEGY_BITS > globalStateToValueBits (GlobalState *globalState)

Changes globalState variables to a bitset used in natural strategy synthesis.

- vector< tuple< vector< tuple< bool, string > >, string > > reduceStrategy (vector< tuple< vector< tuple< bool, string > >, string > > strategyEntries, short lockedColumn=0, bool upperHalf=false)

  Natural strategy reduction algorithm.
- void increaseProbability (GlobalState *currentState, GlobalTransition *decision)

  Increases next GlobalState probability reached through the decision by adding to it a multiplied currentState probability with the decision probability.
- void lowerProbability (GlobalState *currentStateFrom, GlobalState *currentStateTo, set< LocalTransition * > decision)

  Decreases previous GlobalState probability reached from the decision by subtracting from it a multiplied currentState probability with the decision probability.

Protected Attributes

- TraversalMode **mode**

  Current mode of model traversal.
- GlobalState * **revertToGlobalState**

  Global state to which revert will rollback to.
- stack< HistoryEntry * > **historyToRestore**

  A history of decisions to be rolled back.
- GlobalModelGenerator * **generator**

  Holds current model and formula.
- HistoryEntry * **historyStart**

  Pointer to the start of model traversal history.
- HistoryEntry * **historyEnd**

  Pointer to the end of model traversal history.
- map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > **naturalStrategy**

  A table of actions paired vith globalState internal variables state for natural strategy construction.
- short **strategyVariableLimit**

  Max strategy variables found.
- vector< string > **variableNames**

  Easily readable variable names for natural strategy generation.
- int **reductionComplexityBefore**

  Natural strategy complexity before reduction.

5.62.1 Detailed Description

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

5.62.2 Constructor & Destructor Documentation

5.62.2.1 Verification()

```
Verification::Verification (
            GlobalModelGenerator * generator )
```

Constructor for Verification.

Parameters

generator	Pointer to GlobalModelGenerator

5.62.3 Member Function Documentation

5.62.3.1 addHistoryContext()

```
bool Verification::addHistoryContext (
            GlobalState * globalState,
            int depth,
            GlobalTransition * decision,
            bool globalTransitionControlled )  [protected]
```

Creates a HistoryEntry of the type CONTEXT and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
depth	Depth of the recursion of the validation algorithm.
decision	Pointer to a transition GlobalTransition selected by the algorithm.
globalTransitionControlled	True if the GlobalTransition is in the set of global transitions controlled by a coalition and it is not a fixed global transition.

Returns

Returns true if the natural strategy is good for now.

5.62.3.2 addHistoryDecision()

```
void Verification::addHistoryDecision (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type DECISION and puts it on top of the stack of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.62.3.3 addHistoryMarkDecisionAsInvalid()

```
void Verification::addHistoryMarkDecisionAsInvalid (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a transition GlobalTransition selected by the algorithm.

5.62.3.4 addHistoryStateStatus()

```
void Verification::addHistoryStateStatus (
            GlobalState * globalState,
            GlobalStateVerificationStatus prevStatus,
            GlobalStateVerificationStatus newStatus )  [protected]
```

Creates a HistoryEntry of the type STATE_STATUS and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
prevStatus	Previous GlobalStateVerificationStatus to be logged.
newStatus	New GlobalStateVerificationStatus to be logged.

5.62.3.5 addHistoryUncontrolledDecision()

```
void Verification::addHistoryUncontrolledDecision (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type UNCONTROLLED_DECISION and puts it on top of the stack of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.62.3.6 areGlobalStatesInTheSameEpistemicClass()

```
bool Verification::areGlobalStatesInTheSameEpistemicClass (
            GlobalState * globalState1,
            GlobalState * globalState2 )  [protected]
```

Compares two GlobalState and checks if their EpistemicClass is the same.

Parameters

globalState1	Pointer to the first GlobalState.
globalState2	Pointer to the second GlobalState.

Returns

Returns true if the EpistemicClass is the same for both of the GlobalState. Returns false if they are different or at least one of them has no EpistemicClass.

5.62.3.7 checkUncontrolledSet()

```
bool Verification::checkUncontrolledSet (
            set< GlobalTransition * > uncontrolledGlobalTransitions,
            GlobalState * globalState,
            int depth,
            bool hasOmittedTransitions,
            bool mixed = false )  [protected]
```

Verifies if each transition from a given state yields a correct result.

Parameters

uncontrolledGlobalTransitions	A set of global transitions to be checked.
globalState	Currently processed global state.
depth	Current recursion depth.
hasOmittedTransitions	Flag with the information about skipped unneeded transitions.

Returns

Returns true if every transition yields a correct result, false otherwise.

5.62.3.8 equivalentGlobalTransitions()

```
bool Verification::equivalentGlobalTransitions (
            GlobalTransition * globalTransition1,
            GlobalTransition * globalTransition2 )  [protected]
```

Checks if two global transitions are made up of the same local transitions.

Parameters

globalTransition1	First global transition to compare.
globalTransition2	Second global transition to compare.

Returns

True if the two global transitions have the same local transitions, false otherwise.

5.62.3.9 getEpistemicClassForGlobalState()

```
EpistemicClass * Verification::getEpistemicClassForGlobalState (
            GlobalState * globalState )  [protected]
```

Gets the EpistemicClass for the agent in passed GlobalState, i.e. transitions from indistinguishable state from certain other states for an agent to other states.

Parameters

globalState	Pointer to a GlobalState of the model.

Returns

Pointer to the EpistemicClass that a coalition of agents from the formula belong to. If there is no such EpistemicClass, returns false.

5.62.3.10 getNaturalStrategy()

```
map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > Verification::getNatural↩
Strategy ( )
```

Get natural strategy map.

Returns

Map of variable values and action names.

5.62.3.11 getReducedStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > Verification::getReducedStrategy
( )
```

Takes natural strategy from memory and reduces it using reduceStrategy().

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.62.3.12 getStrategyComplexity()

```
int Verification::getStrategyComplexity ( )
```

Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.

Returns

Total sum of the natural strategy complexity before reduction.

5.62.3.13 globalStateToValueBits()

```
bitset< STRATEGY_BITS > Verification::globalStateToValueBits (
            GlobalState * globalState ) [protected]
```

Changes globalState variables to a bitset used in natural strategy synthesis.

Parameters

globalState	State that we want to extractvariable values from.

Returns

Bitset of environment variable values.

5.62.3.14 increaseProbability()

```
void Verification::increaseProbability (
            GlobalState * currentState,
            GlobalTransition * decision )  [protected]
```

Increases next GlobalState probability reached through the decision by adding to it a multiplied currentState probability with the decision probability.

Parameters

currentState	From which GlobalState the action was executed.
decision	GlobalTransition containing all transitions in the executed action and the next reachable GlobalState.

5.62.3.15 isAgentInCoalition()

```
bool Verification::isAgentInCoalition (
            Agent * agent )  [protected]
```

Checks if the Agent is in a coalition based on the formula in a GlobalModelGenerator.

Parameters

agent	Pointer to an Agent that is to be checked.

Returns

Returns true if the Agent is in a coalition, otherwise returns false.

5.62.3.16 isGlobalTransitionControlledByCoalition()

```
bool Verification::isGlobalTransitionControlledByCoalition (
            GlobalTransition * globalTransition )  [protected]
```

Checks if any of the LocalTransition in a given GlobalTransition has an Agent in a coalition in the formula.

Parameters

globalTransition	Pointer to a GlobalTransition in a model.

Returns

Returns true if the Agent is in coalition in the formula, otherwise returns false.

5.62.3.17 lowerProbability()

```
void Verification::lowerProbability (
            GlobalState * currentStateFrom,
            GlobalState * currentStateTo,
            set< LocalTransition * > decision )  [protected]
```

Decreases previous GlobalState probability reached from the decision by subtracting from it a multiplied current↩ State probability with the decision probability.

Parameters

currentStateFrom	From which GlobalState the action was executed.
currentStateTo	To which grobal state the action was executed.
decision	Set of LocalTransition containing all transitions in the executed action.

5.62.3.18 newHistoryMarkDecisionAsInvalid()

```
HistoryEntry * Verification::newHistoryMarkDecisionAsInvalid (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and returns it.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a transition GlobalTransition selected by the algorithm.

Returns

Returns pointer to a new HistoryEntry.

5.62.3.19 printCurrentHistory()

```
void Verification::printCurrentHistory (
            int depth )  [protected]
```

Prints current history to the console.

Parameters

depth	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.

5.62.3.20 reduceStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > Verification::reduceStrategy (
            vector< tuple< vector< tuple< bool, string > >, string > > strategyEntries,
            short lockedColumn = 0,
            bool upperHalf = false )  [protected]
```

Natural strategy reduction algorithm.

Parameters

strategyEntries	Vector of tuples <vector of tuples <variable value, variable name>, action name> containing not reduced natural strategy.
lockedColumn	Index of the furthest leftmost locked column.
upperHalf	Indicates if the first columns of the processed strategy entries can't be shuffled. True if they can't be shuffled, false otherwise.

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.62.3.21 restoreHistory()

```
bool Verification::restoreHistory (
            GlobalState * globalState,
            GlobalTransition * globalTransition,
            int depth,
            bool controlled )  [protected]
```

Restores the decisions made for a given global state and transition in current recursion depth.

Parameters

globalState	Currently processed global state.
globalTransition	Previously selected global transition from the given state to mark as invalid.
depth	Current recursion depth.
controlled	Flag with the information about current type of transition. True if controlled, false if uncontrolled.

Returns

Returns true if current top of the history entires matches with

5.62.3.22 revertLastDecision()

```
bool Verification::revertLastDecision (
            int depth )  [protected]
```

Reverts GlobalState and history to the previous decision state.

Parameters

depth	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.

Returns

Returns true if rollback is successful, otherwise returns false.

5.62.3.23 undoHistoryUntil()

```
void Verification::undoHistoryUntil (
            HistoryEntry * historyEntry,
            bool inclusive,
            int depth )  [protected]
```

Rolls back the history entries up to the certain HistoryEntry.

Parameters

historyEntry	Pointer to a HistoryEntry that the history has to be rolled back to.
inclusive	True if the rollback has to remove the specified entry too.
depth	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.

5.62.3.24 undoLastHistoryEntry()

```
void Verification::undoLastHistoryEntry (
            bool freeMemory )  [protected]
```

Removes the top entry of the history stack.

Parameters

freeMemory	True if the entry has to be removed from memory.

5.62.3.25 verify()

```
bool Verification::verify ( )
```

Starts the process of formula verification on a model.

Returns

Returns true if the verification is PENDING or VERIFIED_OK, otherwise returns false.

5.62.3.26 verifyGlobalState()

```
bool Verification::verifyGlobalState (
            GlobalState * globalState,
            int depth )  [protected]
```

Recursively verifies GlobalState.

Parameters

globalState	Pointer to a GlobalState of the model.
depth	Current depth of the recursion.

Returns

Returns true if the verification is PENDING or VERIFIED_OK, otherwise returns false.

5.62.3.27 verifyLocalStates()

```
bool Verification::verifyLocalStates (
            vector< LocalState * > * localStates,
            GlobalState * globalState )  [protected]
```

Verifies a set of LocalState that a GlobalState is composed of with a hardcoded formula.

Parameters

localStates	A pointer to a set of pointers to LocalState.

Returns

Returns true if there is a LocalState with a specific set of values, fulfilling the criteria, otherwise returns false.

5.62.3.28 verifyStrategy()

```
Result Verification::verifyStrategy ( )
```

Verifies a strategy with the previously given models and formula.

Returns

Result containing data of the verification result.

5.62.3.29 verifyTransitionSets()

```
bool Verification::verifyTransitionSets (
            set< GlobalTransition * > controlledGlobalTransitions,
            set< GlobalTransition * > uncontrolledGlobalTransitions,
            GlobalState * globalState,
            int depth,
            bool hasOmittedTransitions,
            bool isFMode,
            bool mixedTransitions = false )  [protected]
```

Checks if given transition sets are able to fulfill the formula for its given epistemic class.

Parameters

controlledGlobalTransitions	Set of controlled transitions in the current global state.
uncontrolledGlobalTransitions	Set of uncontrolled transitions in the current global state.
globalState	Currently processed global state.
depth	Current recursion depth.
hasOmittedTransitions	Flag with the information about skipped unneeded transitions.

Returns

True if there is a correct choice for an agent to take, false otherwise.

The documentation for this class was generated from the following files:

- Verification.hpp
- Verification.cpp

5.63 VerificationIterative Class Reference

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

```
#include <VerificationIterative.hpp>
```

Public Member Functions

- VerificationIterative (GlobalModelGenerator *generator)
  Constructor for Verification.
- ~**VerificationIterative** ()
  Destructor for Verification.
- map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > getNaturalStrategy ()
  Get natural strategy map.
- vector< tuple< vector< tuple< bool, string > >, string > > getReducedStrategy ()
  Takes natural strategy from memory and reduces it using reduceStrategy().
- int getStrategyComplexity ()
  Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.
- Result verify ()
  Verifies a strategy with the previously given models and formula.

Protected Member Functions

- bool verifyLocalStates (vector< LocalState * > *localStates, GlobalState *globalState)

  Verifies a set of LocalState that a GlobalState is composed of with a hardcoded formula.
- bool isGlobalTransitionControlledByCoalition (GlobalTransition *globalTransition)

  Checks if any of the LocalTransition in a given GlobalTransition has an Agent in a coalition in the formula.
- bool isAgentInCoalition (Agent *agent)

  Checks if the Agent is in a coalition based on the formula in a GlobalModelGenerator.
- EpistemicClass * getEpistemicClassForGlobalState (GlobalState *globalState)

  Gets the EpistemicClass for the agent in passed GlobalState, i.e. transitions from indistinguishable state from certain other states for an agent to other states.
- bool areGlobalStatesInTheSameEpistemicClass (GlobalState *globalState1, GlobalState *globalState2)

  Compares two GlobalState and checks if their EpistemicClass is the same.
- string checkIfProbabilistic (GlobalTransition *transitionToCheck)

  Checks if a given transition is a probabilistic one.
- void findLoopedProbabilityTransitions (map< string, set< GlobalTransition * > > *transitionSets)

  Looks for the probability transitions with loops and dissolves them.
- void dissolveLoopedProbabilityTransition (GlobalTransition *transitionToDissolve, set< GlobalTransition * > *probabilityTransitions)

  Marks a loop as a transition to dissolve.
- void addHistoryDecision (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type DECISION and puts it on top of the stack of the decision history.
- void addHistoryStateStatus (GlobalState *globalState, GlobalStateVerificationStatus prevStatus, GlobalStateVerificationStatus newStatus)

  Creates a HistoryEntry of the type STATE_STATUS and puts it to the top of the decision history.
- bool addHistoryContext (GlobalState *globalState, int depth, GlobalTransition *decision, bool globalTransitionControlled)

  Creates a HistoryEntry of the type CONTEXT and puts it to the top of the decision history.
- void **addHistoryProbability** (GlobalTransition *decision)
- void addHistoryAnswerProbability (StateVerificationInfo *globalState, ProbabilityCalculationType currentMode, ProbabilityTrueFalse newProbabilityValues)

  Adds a history entry for the change of probabilistic answer in a particular state.
- void addHistoryMarkDecisionAsInvalid (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.
- bool revertToLastDecision ()

  Reverts the taken actions to the paint of the last decision.
- void **undoLastHistoryEntry** ()

  Removes the top entry of the history stack.
- bool equivalentGlobalTransitions (GlobalTransition *globalTransition1, GlobalTransition *globalTransition2)

  Checks if two global transitions are made up of the same local transitions.
- bitset< STRATEGY_BITS > globalStateToValueBits (GlobalState *globalState)

  Changes globalState variables to a bitset used in natural strategy synthesis.
- vector< tuple< vector< tuple< bool, string > >, string > > reduceStrategy (vector< tuple< vector< tuple< bool, string > >, string > > strategyEntries, short lockedColumn=0, bool upperHalf=false)

  Natural strategy reduction algorithm.
- bool testForAndFixBadAgents (StateVerificationInfo *stateVerificationInfo)

  Fixes potentially blocking agent actions.

Protected Attributes

- GraphTraversalMode **mode** = GraphTraversalMode::NORMAL_TRAVERSAL

  Current mode of model traversal.
- GlobalState * **revertToGlobalState**

  Global state to which revert will rollback to.
- GlobalModelGenerator * **generator**

  Holds current model and formula.
- stack< DecisionEntry > **history**

  Stack of decisions made.
- map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > **naturalStrategy**

  A table of actions paired vith globalState internal variables state for natural strategy construction.
- short **strategyVariableLimit**

  Max strategy variables found.
- vector< string > **variableNames**

  Easily readable variable names for natural strategy generation.
- int **reductionComplexityBefore**

  Natural strategy complexity before reduction.
- stack< StateVerificationInfo > **statesToProcess**

5.63.1 Detailed Description

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

5.63.2 Constructor & Destructor Documentation

5.63.2.1 VerificationIterative()

```
VerificationIterative::VerificationIterative (
            GlobalModelGenerator * generator )
```

Constructor for Verification.

Parameters

generator	Pointer to GlobalModelGenerator

5.63.3 Member Function Documentation

5.63.3.1 addHistoryAnswerProbability()

```
void VerificationIterative::addHistoryAnswerProbability (
            StateVerificationInfo * stateVerifInfo,
            ProbabilityCalculationType currentMode,
            ProbabilityTrueFalse newProbabilityValues )  [protected]
```

Adds a history entry for the change of probabilistic answer in a particular state.

Parameters

globalState	
currentMode	SUM_PROBABILITY if controlled, MIN_PROBABILITY if uncontrolled.
newProbabilityValues	

5.63.3.2 addHistoryContext()

```
bool VerificationIterative::addHistoryContext (
            GlobalState * globalState,
            int depth,
            GlobalTransition * decision,
            bool globalTransitionControlled )  [protected]
```

Creates a HistoryEntry of the type CONTEXT and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
depth	Depth of the recursion of the validation algorithm.
decision	Pointer to a transition GlobalTransition selected by the algorithm.
globalTransitionControlled	True if the GlobalTransition is in the set of global transitions controlled by a coalition and it is not a fixed global transition.

Returns

Returns true if the natural strategy is good for now.

5.63.3.3 addHistoryDecision()

```
void VerificationIterative::addHistoryDecision (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type DECISION and puts it on top of the stack of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.63.3.4 addHistoryMarkDecisionAsInvalid()

```
void VerificationIterative::addHistoryMarkDecisionAsInvalid (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a transition GlobalTransition selected by the algorithm.

5.63.3.5 addHistoryStateStatus()

```
void VerificationIterative::addHistoryStateStatus (
            GlobalState * globalState,
            GlobalStateVerificationStatus prevStatus,
            GlobalStateVerificationStatus newStatus )  [protected]
```

Creates a HistoryEntry of the type STATE_STATUS and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
prevStatus	Previous GlobalStateVerificationStatus to be logged.
newStatus	New GlobalStateVerificationStatus to be logged.

5.63.3.6 areGlobalStatesInTheSameEpistemicClass()

```
bool VerificationIterative::areGlobalStatesInTheSameEpistemicClass (
            GlobalState * globalState1,
            GlobalState * globalState2 )  [protected]
```

Compares two GlobalState and checks if their EpistemicClass is the same.

Parameters

globalState1	Pointer to the first GlobalState.
globalState2	Pointer to the second GlobalState.

Returns

Returns true if the EpistemicClass is the same for both of the GlobalState. Returns false if they are different or at least one of them has no EpistemicClass.

5.63.3.7 checkIfProbabilistic()

```
string VerificationIterative::checkIfProbabilistic (
            GlobalTransition * transitionToCheck )  [protected]
```

Checks if a given transition is a probabilistic one.

Parameters

transitionToCheck	Transition to be checked if posesses probability != 1.

Returns

Returns concatenated names of local transitions separated with semicolons if true, returns empty string otherwise.

5.63.3.8 dissolveLoopedProbabilityTransition()

```
void VerificationIterative::dissolveLoopedProbabilityTransition (
            GlobalTransition * transitionToDissolve,
            set< GlobalTransition * > * probabilityTransitions )  [protected]
```

Marks a loop as a transition to dissolve.

Parameters

transitionToDissolve	GlobalTransition to ignore.
probabilityTransitions	Set of GlobalTransitions to remove the transition from (unused)

5.63.3.9 equivalentGlobalTransitions()

```
bool VerificationIterative::equivalentGlobalTransitions (
            GlobalTransition * globalTransition1,
            GlobalTransition * globalTransition2 )  [protected]
```

Checks if two global transitions are made up of the same local transitions.

Parameters

globalTransition1	First global transition to compare.
globalTransition2	Second global transition to compare.

Returns

True if the two global transitions have the same local transitions, false otherwise.

5.63.3.10 findLoopedProbabilityTransitions()

```
void VerificationIterative::findLoopedProbabilityTransitions (
            map< string, set< GlobalTransition * > > * transitionSets )  [protected]
```

Looks for the probability transitions with loops and dissolves them.

Parameters

transitionSets	Set of GlobalTransitions to check for the loops.

5.63.3.11 getEpistemicClassForGlobalState()

```
EpistemicClass * VerificationIterative::getEpistemicClassForGlobalState (
            GlobalState * globalState )  [protected]
```

Gets the EpistemicClass for the agent in passed GlobalState, i.e. transitions from indistinguishable state from certain other states for an agent to other states.

Parameters

globalState	Pointer to a GlobalState of the model.

Returns

Pointer to the EpistemicClass that a coalition of agents from the formula belong to. If there is no such EpistemicClass, returns false.

5.63.3.12 getNaturalStrategy()

```
map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > VerificationIterative::get↩
NaturalStrategy ( )
```

Get natural strategy map.

Returns

Map of variable values and action names.

5.63.3.13 getReducedStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > VerificationIterative::get↩
ReducedStrategy ( )
```

Takes natural strategy from memory and reduces it using reduceStrategy().

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.63.3.14 getStrategyComplexity()

```
int VerificationIterative::getStrategyComplexity ( )
```

Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.

Returns

Total sum of the natural strategy complexity before reduction.

5.63.3.15 globalStateToValueBits()

```
bitset< STRATEGY_BITS > VerificationIterative::globalStateToValueBits (
            GlobalState * globalState )  [protected]
```

Changes globalState variables to a bitset used in natural strategy synthesis.

Parameters

globalState	State that we want to extractvariable values from.

Returns

Bitset of environment variable values.

5.63.3.16 isAgentInCoalition()

```
bool VerificationIterative::isAgentInCoalition (
            Agent * agent )  [protected]
```

Checks if the Agent is in a coalition based on the formula in a GlobalModelGenerator.

Parameters

agent	Pointer to an Agent that is to be checked.

Returns

Returns true if the Agent is in a coalition, otherwise returns false.

5.63.3.17 isGlobalTransitionControlledByCoalition()

```
bool VerificationIterative::isGlobalTransitionControlledByCoalition (
            GlobalTransition * globalTransition )  [protected]
```

Checks if any of the LocalTransition in a given GlobalTransition has an Agent in a coalition in the formula.

Parameters

globalTransition	Pointer to a GlobalTransition in a model.

Returns

Returns true if the Agent is in coalition in the formula, otherwise returns false.

5.63.3.18 reduceStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > VerificationIterative::reduce↩
Strategy (
            vector< tuple< vector< tuple< bool, string > >, string > > strategyEntries,
            short lockedColumn = 0,
            bool upperHalf = false )  [protected]
```

Natural strategy reduction algorithm.

Parameters

strategyEntries	Vector of tuples <vector of tuples <variable value, variable name>, action name> containing not reduced natural strategy.
lockedColumn	Index of the furthest leftmost locked column.
upperHalf	Indicates if the first columns of the processed strategy entries can't be shuffled. True if they can't be shuffled, false otherwise.

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.63.3.19 revertToLastDecision()

```
bool VerificationIterative::revertToLastDecision ( )  [protected]
```

Reverts the taken actions to the paint of the last decision.

Returns

True if reverted correctly, false otherwise.

5.63.3.20 testForAndFixBadAgents()

```
bool VerificationIterative::testForAndFixBadAgents (
            StateVerificationInfo * stateVerificationInfo )  [protected]
```

Fixes potentially blocking agent actions.

Parameters

stateVerificationInfo	Information for the state verification.

Returns

True if changed transitions into uncontrolled, false otherwise.

5.63.3.21 verify()

```
Result VerificationIterative::verify ( )
```

Verifies a strategy with the previously given models and formula.

Returns

Result containing data of the verification result.

5.63.3.22 verifyLocalStates()

```
bool VerificationIterative::verifyLocalStates (
            vector< LocalState * > * localStates,
            GlobalState * globalState )  [protected]
```

Verifies a set of LocalState that a GlobalState is composed of with a hardcoded formula.

Parameters

localStates	A pointer to a set of pointers to LocalState.

Returns

Returns true if there is a LocalState with a specific set of values, fulfilling the criteria, otherwise returns false.

The documentation for this class was generated from the following files:

- VerificationIterative.hpp
- VerificationIterative.cpp

5.64 yy_buffer_state Struct Reference

Public Attributes

- FILE * **yy_input_file**
- char * **yy_ch_buf**
- char * **yy_buf_pos**

- int **yy_buf_size**
- int **yy_n_chars**
- int **yy_is_our_buffer**
- int **yy_is_interactive**
- int **yy_at_bol**
- int yy_bs_lineno
- int yy_bs_column
- int **yy_fill_buffer**
- int **yy_buffer_status**

5.64.1 Member Data Documentation

5.64.1.1 yy_bs_column

```
int yy_buffer_state::yy_bs_column
```

The column count.

5.64.1.2 yy_bs_lineno

```
int yy_buffer_state::yy_bs_lineno
```

The line count.

The documentation for this struct was generated from the following files:

- scanner.c
- strategyScanner.c

5.65 yy_trans_info Struct Reference

Public Attributes

- flex_int32_t **yy_verify**
- flex_int32_t **yy_nxt**

The documentation for this struct was generated from the following files:

- scanner.c
- strategyScanner.c

5.66 yyalloc Union Reference

Public Attributes

- yy_state_t **yyss_alloc**
- YYSTYPE **yyvs_alloc**

The documentation for this union was generated from the following files:

- parser.c
- strategyParser.c

5.67 YYSTYPE Union Reference

Public Attributes

- set< class AgentTemplate * > * **model**
- class ExprNode * **expr**
- class ProbNode * **prob**
- class Assignment * **assign**
- class TransitionTemplate * **trans**
- class AgentTemplate * **agent**
- set< class Assignment * > * **assignSet**
- char * **ident**
- set< string > * **identSet**
- int **val**
- float **floatno**
- vector< class ExprNode * > * **condList**

The documentation for this union was generated from the following file:

- parser.h

Chapter 6

File Documentation

6.1 Agent.cpp File Reference

Class of an agent. Class of an agent.

```
#include "Agent.hpp"
#include "LocalState.hpp"
```

6.1.1 Detailed Description

Class of an agent. Class of an agent.

6.2 Agent.hpp File Reference

Class of an agent. Class of an agent.

```
#include "Common.hpp"
```

Classes

- class Agent

  Contains all data for a single Agent, including id, name and all of the agents' variables.

6.2.1 Detailed Description

Class of an agent. Class of an agent.

6.3 Agent.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef AGENT_H
00008 #define AGENT_H
00009
00010 #include "Common.hpp"
00011
00013 class Agent {
00014     public:
00016         int id;
00017
00019         string name;
00020
00022         set<Var*> vars;
00023
00027         Agent(int _id, string _name):id(_id), name(_name) {};
00028
00030         LocalState* initState;
00031
00033         vector<LocalState*> localStates; // localStates[i].id == i
00034
00036         vector<LocalTransition*> localTransitions; // localTransitions[i].id == i
00037
00038         // sprawdź, czy stan nie został już wygenerowany.
00039
00040         LocalState* includesState(LocalState *state);
00041 };
00042
00043 #endif // AGENT_H
```

6.4 Common.hpp File Reference

Contains all commonly used classes and structs. Contains all commonly used classes and structs.

```
#include <map>
#include <set>
#include <stack>
#include <string>
#include <utility>
#include <vector>
#include <atomic>
#include "reader/expressions.hpp"
#include "enums/GlobalStateVerificationStatus.hpp"
#include "enums/ConditionOperator.hpp"
```

Classes

- struct Cfg

6.4.1 Detailed Description

Contains all commonly used classes and structs. Contains all commonly used classes and structs.

6.5 Common.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef COMMON
00008 #define COMMON
00009
00010 #include <map>
00011 #include <set>
00012
00013 #include <stack>
00014 #include <string>
00015 #include <utility>
00016 #include <vector>
00017 #include <atomic> //std::atomic_uint32_t
00018
00019 #include "reader/expressions.hpp"
00020
00021 #include "enums/GlobalStateVerificationStatus.hpp"
00022 #include "enums/ConditionOperator.hpp"
00023
00024 class Agent;
00025 class LocalState;
00026
00027 struct Condition;
00028 struct EpistemicClass;
00029 struct Formula;
00030 struct GlobalModel;
00031 struct GlobalState;
00032 struct GlobalTransition;
00033 struct LocalTransition;
00034 struct LocalModels;
00035 struct Var;
00036
00037 struct Cfg{
00038     std::string fname;
00039     int stv_mode;
00040     bool output_local_models;
00041     bool output_global_model;
00042     bool output_dot_files;
00043     std::string dotdir;
00044     int model_id; // <-- this is temporary member (used in Verification.cpp for a hardcoded formula);
      has lower priority than `fname`
00045     bool add_epsilon_transitions;
00046     bool formula_from_parameter;
00047     std::string formula;
00048     bool counterexample;
00049     bool reduce;
00050     bool reduce_all;
00051     std::string reduce_args;
00052     bool fixpoint;
00053     bool natural_strategy;
00054     bool probability = false;
00055     bool verify_strategy;
00056     std::string strategy_file_path;
00057 };
00058
00059 #endif
```

6.6 Constants.hpp

```
00001 #ifndef SELENE_CONSTANTS
00002 #define SELENE_CONSTANTS
00003
00004 #define VERBOSE 0
00005 #define DEBUG_ON 0
00006 // #define OUTPUT_LOCAL_MODELS 1
00007 // #define OUTPUT_GLOBAL_MODEL 0 // warning: it will call expandAllStates()
00008 // #define MODE 2 // 1 = only generate; 2 = verify
00009
00010 // Model id
00011 // 1 = /examples/trains/Trains2Controller1.txt
00012 // 2 = /examples/ssvr/Selene_Select_Vote_Revoting_1v_1cv_3c_3rev_share.txt
00013 // 3 = /examples/svote/Voters2Candidates2.txt
00014 // #define MODEL_ID 1
00015
00016 #endif // SELENE_CONSTANTS
```

6.7 DotGraph.cpp File Reference

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

```
#include "DotGraph.hpp"
#include <algorithm>
```

6.7.1 Detailed Description

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

6.8 DotGraph.hpp File Reference

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

```
#include "Types.hpp"
#include "Utils.hpp"
#include "reader/nodes.hpp"
#include <string>
```

Classes

- class DotGraph

Enumerations

- enum DotGraphBase { LOCAL_MODEL , GLOBAL_MODEL , AGENT_TEMPLATE }

6.8.1 Detailed Description

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

6.8.2 Enumeration Type Documentation

6.8.2.1 DotGraphBase

```
enum DotGraphBase
```

Enumerator

LOCAL_MODEL	(unfolded) local model = TS with states and transitions
GLOBAL_MODEL	(unfolded) global model = TS with states and transitions
AGENT_TEMPLATE	(folded/compact) agent graph = sets of locations and labelled edges

6.9 DotGraph.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef DOT_PARSER
00008 #define DOT_PARSER
00009
00010 #include "Types.hpp"
00011 #include "Utils.hpp"
00012 #include "reader/nodes.hpp"
00013 #include <string>
00014
00015 using namespace std;
00016
00017 enum DotGraphBase {
00018     LOCAL_MODEL,
00019     GLOBAL_MODEL,
00020     AGENT_TEMPLATE,
00021 };
00022
00023 class DotGraph{
00024 public:
00025     std::vector<std::string> nodes;
00026     std::vector<std::string> edges;
00027     std::string graphName;
00028     DotGraphBase graphBase;
00029     static string styleString;
00030     DotGraph();
00031     DotGraph(GlobalModel *const gm, bool extended=false, bool correct=false);
00032     DotGraph(Agent *const ag, bool extended=false);
00033     DotGraph(AgentTemplate *const at);
00034     void saveToFile(std::string pathprefix, std::string nameprefix, std::string basename="");
00035 protected:
00036     void addNode(std::string id, std::string name);
00037     void addEdge(std::string src, std::string trg, std::string label);
00038 };
00039
00040 #endif
```

6.10 ConditionOperator.hpp

```
00001 #ifndef CONDITIONOPERATOR_H
00002 #define CONDITIONOPERATOR_H
00003
00005 enum ConditionOperator {
00006     Equals,
00007     NotEquals,
00008 };
00009
00010 #endif // CONDITIONOPERATOR_H
```

6.11 GlobalStateVerificationStatus.hpp

```
00001 #ifndef GLOBAL_STATE_VERIFICATION_STATUS
00002 #define GLOBAL_STATE_VERIFICATION_STATUS
00003
00004 enum GlobalStateVerificationStatus {
00005     UNVERIFIED,
00006     PENDING,
00007     VERIFIED_OK,
00008     VERIFIED_ERR,
00009 };
00010
00011 #endif
```

6.12 EpistemicClass.hpp File Reference

Struct of an epistemic class. Struct of an epistemic class.

`#include "Common.hpp"`

Classes

- struct EpistemicClass

  Represents a single epistemic class.

6.12.1 Detailed Description

Struct of an epistemic class. Struct of an epistemic class.

6.13 EpistemicClass.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef EPISTEMICCLASS_H
00008 #define EPISTEMICCLASS_H
00009
00010 #include "Common.hpp"
00011
00013 struct EpistemicClass {
00015     string hash;
00016
00018     map<string, GlobalState*> globalStates;
00019     // GlobalState->hash => GlobalState*
00020
00022     GlobalTransition* fixedCoalitionTransition;
00023 };
00024
00025 #endif // EPISTEMICCLASS_H
```

6.14 GlobalModel.hpp File Reference

Struct of a global model. Struct of a global model.

`#include "Common.hpp"`

Classes

- struct GlobalModel

  Represents a global model, containing agents and a formula.

6.14.1 Detailed Description

Struct of a global model. Struct of a global model.

6.15 GlobalModel.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef GLOBALMODEL_H
00008 #define GLOBALMODEL_H
00009
00010 #include "Common.hpp"
00011
00013 struct GlobalModel {
00014     // Data
00015
00017     vector<Agent*> agents;
00018     // agents[i].id == i
00019
00021     Formula* formula;
00022
00023     // Bindings
00024
00026     GlobalState* initState;
00027
00029     vector<GlobalState*> globalStates;
00030     // globalStates[i].id == i
00031
00033     map<Agent*, map<string, EpistemicClass*>> epistemicClasses;
00034
00036     map<Agent*, map<string, set<GlobalState*>>> epistemicClassesKnowledge;
00037     // Agent* => (EpistemicClass->hash => EpistemicClass*)
00038 };
00039
00040
00041 #endif // GLOBALMODEL_H
```

6.16 GlobalModelGenerator.cpp File Reference

Generator of a global model. Class for initializing and generating a global model.

```
#include "GlobalModelGenerator.hpp"
#include "Types.hpp"
#include "Constants.hpp"
#include <algorithm>
#include <string.h>
#include <iostream>
#include <sstream>
#include <functional>
#include "craam/RMDP.hpp"
#include "craam/algorithms/values.hpp"
#include "craam/modeltools.hpp"
```

Variables

- Cfg **config**

6.16.1 Detailed Description

Generator of a global model. Class for initializing and generating a global model.

6.17 GlobalModelGenerator.hpp File Reference

Generator of a global model. Class for initializing and generating a global model.

```
#include "Constants.hpp"
#include "GlobalState.hpp"
#include "GlobalTransition.hpp"
#include "Agent.hpp"
#include "Types.hpp"
#include <unordered_set>
#include <unordered_map>
#include <sstream>
#include "craam/RMDP.hpp"
#include "craam/algorithms/values.hpp"
#include "craam/modeltools.hpp"
```

Classes

- class GlobalModelGenerator

  Stores the local models, formula and a global model.
- struct GlobalModelGenerator::GlobalStateTupleHash

6.17.1 Detailed Description

Generator of a global model. Class for initializing and generating a global model.

6.18 GlobalModelGenerator.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef SELENE_GLOBAL_MODEL_GENERATOR
00008 #define SELENE_GLOBAL_MODEL_GENERATOR
00009
00010 #include "Constants.hpp"
00011 #include "GlobalState.hpp"
00012 #include "GlobalTransition.hpp"
00013 #include "Agent.hpp"
00014 #include "Types.hpp"
00015
00016 #include <unordered_set>
00017 #include <unordered_map>
00018 #include <sstream>
00019
00020 #include "craam/RMDP.hpp"
00021 #include "craam/algorithms/values.hpp"
00022 #include "craam/modeltools.hpp"
00023
00024 using namespace std;
00025 using namespace craam;
00026
00028 class GlobalModelGenerator {
00029 public:
00030     GlobalModelGenerator();
00031     ~GlobalModelGenerator();
00032     GlobalState* initModel(LocalModels* localModels, Formula* formula);
00033     void expandState(GlobalState* state);
00034     vector<GlobalState*> expandStateAndReturn(GlobalState* state, bool returnAnyway = false);
00035     void expandAllStates(bool additionalProbSplit = false);
00036     void expandAndReduceAllStates();
00037     GlobalModel* getCurrentGlobalModel();
00038     Formula* getFormula();
```

```
00039     int getFormulaSize();
00040     set<GlobalState*>* findOrCreateEpistemicClassForKnowledge(vector<LocalState*>* localStates,
      GlobalState* globalState, Agent* agent);
00041     Agent* getAgentInstanceByName(string agentName);
00042     void markFormulaAsIncorrect();
00043     bool getFormulaCorectness();
00044     void initStrategy(StrategyCollection* strat);
00045     set<set<tuple<string, string>> createProbabilityStrategy(LocalModels* localModels);
00046     set<tuple<string, string>* getNextPath();  // Returns next strategy iteratively (one per call)
00047     MDP generateNextMDP(bool makeOpponentGoMax = false);
00048     string getCoalitionIdentifier(vector<LocalState *> *localStates);
00049     // Returns coalition-only action signature, ordered by agentIndex and using localName when
      available
00050     string getCoalitionActionSignature(GlobalTransition* transition, char sep=';');
00051     string getActionNameFromStateInStrategy(GlobalState* state);
00052
00054     map<Agent*,size_t> agentIndex;
00055
00056 protected:
00057     struct GlobalStateTupleHash {
00058         size_t operator()(const tuple<GlobalState*, int>& t) const {
00059             auto gsHash = hash<string>()(get<0>(t)->hash);
00060             auto deHash = hash<int>()(get<1>(t));
00061             return gsHash ^ deHash;
00062         }
00063     };
00065     LocalModels* localModels;
00067     Formula* formula;
00069     GlobalModel* globalModel;
00071     bool correctModel;
00073     unordered_map<GlobalState*, unordered_set<int>> candidateStateDepths;
00075     stack<GlobalState*> globalModelCandidates;
00077     stack<int> stateDepths;
00079     unordered_set<GlobalState*> addedStates;
00081     StrategyCollection* strategyCollection;
00082     GlobalState* generateInitState();
00083     GlobalState* generateStateFromLocalStates(vector<LocalState*>* localStates, set<LocalTransition*>*
      viaLocalTransitions, GlobalState* prevGlobalState);
00084     void generateGlobalTransitions(GlobalState* fromGlobalState, set<LocalTransition*>
      localTransitions, map<Agent*, vector<LocalTransition*>> transitionsByAgent);
00085     string computeEpistemicClassHash(vector<LocalState*>* localStates, Agent* agent);
00086     string computeGlobalStateHash(vector<LocalState*>* localStates);
00087     EpistemicClass* findOrCreateEpistemicClass(vector<LocalState*>* localStates, Agent* agent);
00088     GlobalState* findGlobalStateInEpistemicClass(vector<LocalState*>* localStates, EpistemicClass*
      epistemicClass);
00089     set<tuple<string, string>> currentStrategy;  // Current strategy being built
00090
00091     // For iterative strategy generation: track which action choice (0, 1, 2, ...) at each epistemic
      class
00092     map<string, size_t> choiceIndices;  // coalitionId -> which action choice to try (0-indexed)
00093     map<string, size_t> actionCounts;   // coalitionId -> total number of actions available
00094     bool strategyGenerationInit = false;
00095     bool strategiesExhausted = false;
00096
00097     // Cache for efficient state hash lookups
00098     unordered_map<string, GlobalState*> stateHashMapCache;
00099
00100     map<string, map<string, set<GlobalTransition*>>> coalitionTransitions; // state, actionName, actual
      transitions
00101     map<string, map<string, set<GlobalTransition*>>> opponentsTransitions; // state, actionName, actual
      transitions
00102     bool checkLocalStates(vector<LocalState*>* localStates, GlobalState* globalState);
00103 };
00104
00105 #endif // SELENE_GLOBAL_MODEL_GENERATOR
```

6.19 GlobalState.cpp File Reference

Struct representing a global state. Struct representing a global state.

```
#include "GlobalState.hpp"
#include "LocalState.hpp"
```

6.19.1 Detailed Description

Struct representing a global state. Struct representing a global state.

6.20 GlobalState.hpp File Reference

Struct representing a global state. Struct representing a global state.

```
#include "Common.hpp"
#include "TypesDependency.hpp"
```

Classes

- struct GlobalState

  Represents a single global state.

Variables

- Cfg **config**

6.20.1 Detailed Description

Struct representing a global state. Struct representing a global state.

6.21 GlobalState.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef GLOBAL_STATE
00008 #define GLOBAL_STATE
00009
00010 #include "Common.hpp"
00011 #include "TypesDependency.hpp"
00012
00013 extern Cfg config;
00014
00016 struct GlobalState {
00017     GlobalState();
00018     // Data
00019
00021     string hash;
00022
00024     map<Agent*, EpistemicClass*> epistemicClasses;
00025
00027     bool isExpanded;
00028
00030     GlobalStateVerificationStatus verificationStatus = GLOBAL_STATE_VERIFICATION_STATUS::UNVERIFIED;
00031
00033     float probability = config.probability ? 0.0 : 1.0;
00034
00035     bool goodState = false;
00036
00037     // Bindings
00038
00040     set<GlobalTransition*> globalTransitions;
00041
00043     set<GlobalState*> preimage;
00044
00046     vector<LocalState*> localStatesProjection;
00047
00049     map<Agent*, set<GlobalState*>*> epistemicClassesAllAgents;
00050
00051     ProbabilityEntry probabilityResult;
00052
00054     GlobalTransition* probabilityLoop = nullptr;
00055
00059     std::string toString(string indent="");
00060
00063     map<string, int> getGlobalStateEnvironment();
00064
00065     VerifResult stateVerifResult = VerifResult::NOT_VERIFIED;
00066 };
00067
00068 #endif
```

6.22 GlobalTransition.cpp File Reference

Struct representing a global transition. Struct representing a global transition.

```
#include "GlobalTransition.hpp"
#include "LocalTransition.hpp"
```

6.22.1 Detailed Description

Struct representing a global transition. Struct representing a global transition.

6.23 GlobalTransition.hpp File Reference

Struct representing a global transition. Struct representing a global transition.

```
#include "Common.hpp"
```

Classes

- struct GlobalTransition

  Represents a single global transition.

6.23.1 Detailed Description

Struct representing a global transition. Struct representing a global transition.

6.24 GlobalTransition.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef GLOBALTRANSITION_H
00008 #define GLOBALTRANSITION_H
00009
00010 #include "Common.hpp"
00011
00013 struct GlobalTransition {
00014     static atomic_uint32_t next_id; // [YK]: ideally it should be a private member (and possibly of a
      type size_t), but for now this should work just as well
00015     GlobalTransition();
00016     // Data
00017
00019     uint32_t id;
00020
00022     bool isInvalidDecision;
00023
00024     // Bindings
00025
00027     GlobalState* from;
00028
00030     GlobalState* to;
00031
00033     set<LocalTransition*> localTransitions;
00034
00035     string joinLocalTransitionNames(char sep=';');
00036
00037     float getProbability();
00038 };
00039
00040 #endif // GLOBALTRANSITION_H
```

6.25 LocalState.cpp File Reference

Class representing a local state. Class representing a local state.

```
#include "LocalState.hpp"
#include "Agent.hpp"
```

6.25.1 Detailed Description

Class representing a local state. Class representing a local state.

6.26 LocalState.hpp File Reference

Class representing a local state. Class representing a local state.

```
#include "Common.hpp"
```

Classes

- class LocalState

  Represents a single LocalState, containing id, name and internal variables.

6.26.1 Detailed Description

Class representing a local state. Class representing a local state.

6.27 LocalState.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef LOCALSTATE_H
00008 #define LOCALSTATE_H
00009
00010 #include "Common.hpp"
00011
00013 class LocalState {
00014     public:
00015         // Data
00016
00018         uint32_t id;
00019
00021         string name;
00022
00024         map<string, int> environment;
00025
00026         // komparator
00027
00028         bool compare(LocalState *state);
00029
00030         // Bindings
00031
00033         Agent* agent;
00034
00036         set<LocalTransition*> localTransitions;
00037
00039         set<GlobalState*> epistemicGlobalStates;
00040
00044         string toString(string indent="");
00045 };
00046
00047 #endif // LOCALSTATE_H
```

6.28 LocalTransition.hpp File Reference

Class representing a local transition. Class representing a local transition.

```
#include "Common.hpp"
```

Classes

- struct LocalTransition

  Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

6.28.1 Detailed Description

Class representing a local transition. Class representing a local transition.

6.29 LocalTransition.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef LOCALTRANSITION_H
00008 #define LOCALTRANSITION_H
00009
00010 #include "Common.hpp"
00011
00013 struct LocalTransition {
00014     // Data
00015
00017     int id;
00018
00020     string name;
00021
00023     string localName;
00024     // if empty => same as name
00025
00027     bool isShared;
00028
00030     int sharedCount;
00031
00033     set<Condition*> conditions;
00034
00036     float probability;
00037
00038     // Bindings
00039
00041     Agent* agent;
00042
00044     LocalState* from;
00045
00047     LocalState* to;
00048 };
00049
00050
00051 #endif // LOCALTRANSITION_H
```

6.30 ModelParser.cc File Reference

A model parser. A parser for converting a text file into a model.

```
#include "ModelParser.hpp"
#include "reader/nodes.hpp"
#include <stdio.h>
#include <string.h>
#include <tuple>
#include <iostream>
```

Functions

- int **yyparse** ()
- void **yyrestart** (FILE ∗)
- void **set_input_string** (const char ∗in)

Variables

- set< AgentTemplate ∗ > ∗ **modelDescription**
- FormulaTemplate **formulaDescription**

6.30.1 Detailed Description

A model parser. A parser for converting a text file into a model.

6.31 ModelParser.hpp

```
00001
00007 #ifndef __TESTPARSER_HPP
00008 #define __TESTPARSER_HPP
00009
00010 #include "Types.hpp"
00011 #include <tuple>
00012
00013 using namespace std;
00014
00016 class ModelParser {
00017 public:
00018     ModelParser();
00019     ~ModelParser();
00020     tuple<LocalModels, Formula> parse(string fileName);
00021     tuple<LocalModels, Formula> parseAndOverwriteFormula(string fileName, string s);
00022
00023 protected:
00024     // @internal
00025 };
00026
00027 #endif
```

6.32 expressions.cc File Reference

Eval and helper class for expressions. Eval and helper class for expressions.

```
#include "expressions.hpp"
#include "../GlobalModelGenerator.hpp"
#include <bits/stdc++.h>
```

6.32.1 Detailed Description

Eval and helper class for expressions. Eval and helper class for expressions.

6.33 expressions.hpp File Reference

Eval and helper class for expressions. Eval and helper class for expressions.

```
#include <string>
#include <map>
#include <vector>
```

Classes

- class ExprNode
  Base node for expressions.
- class ExprConst
  Node for a constant.
- class ExprIdent
  Node for an identifier.
- class ExprAdd
  Node for addition.
- class ExprSub
  Node for subtraction.
- class ExprMul
  Node for multiplication.
- class ExprDiv
  Node for division.
- class ExprRem
  Node for modulo.
- class ExprAnd
  Node for AND operator.
- class ExprOr
  Node for OR operator.
- class ExprNot
  Node for NOT operator.
- class ExprEq
  Node for "==" operator.
- class ExprNe
  Node for "!=" operator.
- class ExprLt
  Node for "<" operator.
- class ExprLe
  Node for "<=" operator.
- class ExprGt
  Node for ">" operator.
- class ExprGe
  Node for ">=" operator.
- class ExprKnow
  Node for a knowledge operator.
- class ExprHart
  Node for a Hartley operator.
- class ProbNode

Base node for probability calculations.

- class ProbConst

  Node for a constant.

- class ProbAdd

  Node for addition.

- class ProbSub

  Node for subtraction.

- class ProbMul

  Node for multiplication.

- class ProbDiv

  Node for division.

Typedefs

- typedef map< string, int > **Environment**

  Variable names with their values.

6.33.1 Detailed Description

Eval and helper class for expressions. Eval and helper class for expressions.

6.34 expressions.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef __EXPRESSIONS_H
00008 #define __EXPRESSIONS_H
00009
00010 #include <string>
00011 #include <map>
00012 #include <vector>
00013
00014 using namespace std;
00015
00017 typedef map<string, int> Environment;
00018
00019 class GlobalModelGenerator;
00020 struct GlobalState;
00021
00022 // węzeł bazowy dla wyrażeń
00023
00025 class ExprNode {
00026
00027    public:
00028       // metoda do wyliczenia wartości wyrażenia zależna od typu węzła
00029
00033       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState )
      = 0;
00034       virtual int eval( Environment& env ) = 0;
00035 };
00036
00037 // węzeł dla stałej
00038
00040 class ExprConst: public ExprNode {
00041
00042    // argumenty
00043
00045    int val;
00046
00047    public:
00050       ExprConst(int _val): val(_val) {};
00051
00055       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00056       virtual int eval( Environment& env );
```

```
00057 };
00058
00059 // węzeł dla identyfikatora
00060
00062 class ExprIdent: public ExprNode {
00063
00064    // argumenty
00065
00067    string ident;
00068
00069    public:
00072       ExprIdent(string _ident): ident(_ident) {};
00073
00077       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00078       virtual int eval( Environment& env );
00079 };
00080
00081 // węzeł dla dodawań
00082
00084 class ExprAdd: public ExprNode {
00085
00086    // argumenty
00087
00089    ExprNode *larg, *rarg;
00090
00091    public:
00095       ExprAdd(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00096
00100       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00101       virtual int eval( Environment& env );
00102 };
00103
00104 // węzeł dla odejmowań
00105
00107 class ExprSub: public ExprNode {
00108
00109    // argumenty
00110
00112    ExprNode *larg, *rarg;
00113
00114    public:
00118       ExprSub(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00119
00123       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00124       virtual int eval( Environment& env );
00125 };
00126
00127 // węzeł dla mnożeń
00128
00130 class ExprMul: public ExprNode {
00131
00132    // argumenty
00133
00135    ExprNode *larg, *rarg;
00136
00137    public:
00141       ExprMul(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00142
00146       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00147       virtual int eval( Environment& env );
00148 };
00149
00150 // węzeł dla dzieleń
00151
00153 class ExprDiv: public ExprNode {
00154
00155    // argumenty
00156
00158    ExprNode *larg, *rarg;
00159
00160    public:
00164       ExprDiv(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00165
00169       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00170       virtual int eval( Environment& env );
00171 };
00172
00173 // węzeł dla reszty z dzielenia
00174
00176 class ExprRem: public ExprNode {
00177
00178    // argumenty
00179
00181    ExprNode *larg, *rarg;
00182
00183    public:
00187       ExprRem(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
```

```
00188
00192       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00193       virtual int eval( Environment& env );
00194 };
00195
00196 // węzeł dla operatora AND
00197
00199 class ExprAnd: public ExprNode {
00200
00201    // argumenty
00202
00204    ExprNode *larg, *rarg;
00205
00206    public:
00210       ExprAnd(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00211
00215       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00216       virtual int eval( Environment& env );
00217 };
00218
00219 // węzeł dla operatora OR
00220
00222 class ExprOr: public ExprNode {
00223
00224    // argumenty
00225
00227    ExprNode *larg, *rarg;
00228
00229    public:
00233       ExprOr(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00234
00238       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00239       virtual int eval( Environment& env );
00240 };
00241
00242 // węzeł dla operatora NOT
00243
00245 class ExprNot: public ExprNode {
00246
00247    // argumenty
00248
00250    ExprNode *arg;
00251
00252    public:
00255       ExprNot(ExprNode *_arg): arg(_arg) {};
00256
00260       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00261       virtual int eval( Environment& env );
00262 };
00263
00264 // węzeł dla operatora "=="
00265
00267 class ExprEq: public ExprNode {
00268
00269    // argumenty
00270
00272    ExprNode *larg, *rarg;
00273
00274    public:
00278       ExprEq(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00279
00283       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00284       virtual int eval( Environment& env );
00285 };
00286
00287 // węzeł dla operatora "!="
00288
00290 class ExprNe: public ExprNode {
00291
00292    // argumenty
00293
00295    ExprNode *larg, *rarg;
00296
00297    public:
00301       ExprNe(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00302
00306       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00307       virtual int eval( Environment& env );
00308 };
00309
00310 // węzeł dla operatora "<"
00311
00313 class ExprLt: public ExprNode {
00314
00315    // argumenty
00316
00318    ExprNode *larg, *rarg;
```

```
00319
00320    public:
00324       ExprLt(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00325
00329       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00330       virtual int eval( Environment& env );
00331 };
00332
00333 // węzeł dla operatora "<="
00334
00336 class ExprLe: public ExprNode {
00337
00338    // argumenty
00339
00341    ExprNode *larg, *rarg;
00342
00343    public:
00347       ExprLe(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00348
00352       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00353       virtual int eval( Environment& env );
00354 };
00355
00356 // węzeł dla operatora ">"
00357
00359 class ExprGt: public ExprNode {
00360
00361    // argumenty
00362
00364    ExprNode *larg, *rarg;
00365
00366    public:
00370       ExprGt(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00371
00375       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00376       virtual int eval( Environment& env );
00377 };
00378
00379 // węzeł dla operatora ">="
00380
00382 class ExprGe: public ExprNode {
00383
00384    // argumenty
00385
00387    ExprNode *larg, *rarg;
00388
00389    public:
00393       ExprGe(ExprNode *_larg, ExprNode *_rarg): larg(_larg), rarg(_rarg) {};
00394
00398       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00399       virtual int eval( Environment& env );
00400 };
00401
00402 // węzeł dla wiedzy
00403
00405 class ExprKnow: public ExprNode {
00406
00407    // argumenty
00408
00410    string agentName;
00411    ExprNode *arg;
00412
00413    public:
00417       ExprKnow(string _agentName, ExprNode *_arg): agentName(_agentName), arg(_arg) {};
00418
00422       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00423       virtual int eval( Environment& env );
00424 };
00425
00426 // węzeł dla Hartleya
00427
00429 class ExprHart: public ExprNode {
00430
00431    // argumenty
00432
00434    string agentName;
00435    bool le;
00436    int val;
00437    vector<ExprNode*>* arg;
00438
00439    public:
00445       ExprHart(string _agentName, bool _le, int _val, vector<ExprNode*>* _arg): agentName(_agentName),
      le(_le), val(_val), arg(_arg) {};
00446
00450       virtual int eval( Environment& env, GlobalModelGenerator *generator, GlobalState *globalState );
00451       virtual int eval( Environment& env );
00452 };
```

```
00453
00455 class ProbNode {
00456    public:
00459       virtual float eval( GlobalModelGenerator *generator, GlobalState *globalState ) = 0;
00460       virtual float eval() = 0;
00461 };
00462
00464 class ProbConst: public ProbNode {
00466    float val;
00467
00468    public:
00471       ProbConst(float _val): val(_val) {};
00472
00476       virtual float eval( GlobalModelGenerator *generator, GlobalState *globalState );
00477       virtual float eval();
00478 };
00479
00481 class ProbAdd: public ProbNode {
00483    ProbNode *larg, *rarg;
00484
00485    public:
00489       ProbAdd(ProbNode *_larg, ProbNode *_rarg): larg(_larg), rarg(_rarg) {};
00490
00494       virtual float eval( GlobalModelGenerator *generator, GlobalState *globalState );
00495       virtual float eval();
00496 };
00497
00499 class ProbSub: public ProbNode {
00501    ProbNode *larg, *rarg;
00502
00503    public:
00507       ProbSub(ProbNode *_larg, ProbNode *_rarg): larg(_larg), rarg(_rarg) {};
00508
00512       virtual float eval( GlobalModelGenerator *generator, GlobalState *globalState );
00513       virtual float eval();
00514 };
00515
00517 class ProbMul: public ProbNode {
00519    ProbNode *larg, *rarg;
00520
00521    public:
00525       ProbMul(ProbNode *_larg, ProbNode *_rarg): larg(_larg), rarg(_rarg) {};
00526
00530       virtual float eval( GlobalModelGenerator *generator, GlobalState *globalState );
00531       virtual float eval();
00532 };
00533
00535 class ProbDiv: public ProbNode {
00537    ProbNode *larg, *rarg;
00538
00539    public:
00543       ProbDiv(ProbNode *_larg, ProbNode *_rarg): larg(_larg), rarg(_rarg) {};
00544
00548       virtual float eval( GlobalModelGenerator *generator, GlobalState *globalState );
00549       virtual float eval();
00550 };
00551
00552 #endif
```

6.35 nodes.cc File Reference

Parser templates. Class for setting up a new objects from a parser.

```
#include "expressions.hpp"
#include "nodes.hpp"
#include <queue>
#include <fstream>
#include <cstring>
```

6.35.1 Detailed Description

Parser templates. Class for setting up a new objects from a parser.

6.36 nodes.hpp File Reference

Parser templates. Class for setting up a new objects from a parser.

```
#include <string>
#include <set>
#include <map>
#include "expressions.hpp"
#include "../Types.hpp"
```

Classes

- class Assignment

  Represents an assingment.
- class TransitionTemplate

  Represents a meta-transition.
- class LocalStateTemplate

  A template for the local state.
- class AgentTemplate

  Represents a single agent loaded from the description from a file.

6.36.1 Detailed Description

Parser templates. Class for setting up a new objects from a parser.

6.37 nodes.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef __NODES_H
00008 #define __NODES_H
00009
00010 #include <string>
00011 #include <set>
00012 #include <map>
00013 #include "expressions.hpp"
00014
00015 #include "../Types.hpp"
00016
00017 using namespace std;
00018
00019 /* Klasa reprezentująca przypisanie */
00020
00022 class Assignment {
00023    public:
00025       string ident;
00026       // do czego przypisujemy
00027
00029       ExprNode *value;
00030       // co przypisujemy
00031
00035       Assignment(string _ident, ExprNode *_exp): ident(_ident), value(_exp) {};
00036
00037       // wykonaj przypisanie w danym środowisku
00038
00041       virtual void assign(Environment &env) {
00042          env[ident]=value->eval(env);
00043       };
00044 };
00045
00046 /* Klasa reprezentująca meta-tranzycję */
```

```
00047
00049 class TransitionTemplate {
00050    public:
00051       // jeśli -1 to nie ma dzielenia, w p.p. wartość określa łączną liczbę wymaganych agentów
00052
00054       int shared;
00055
00056       // nazwa wzorca
00057
00059       string patternName;
00060
00061       // nazwa do wyszukiwania dla shared
00062
00064       string matchName;
00065
00066       // nazwa stanu początkowego i końcowego
00067
00069       string startState;
00070
00072       string endState;
00073
00074       // wyrażenie warunkowe
00075
00077       ExprNode *condition;
00078
00079       // lista przypisań wartości
00080
00082       set<Assignment*> *assignments;
00083
00085       ProbNode *probability;
00086
00096       TransitionTemplate(int _shared, string _patternName, string _matchName, string _startState,
      string _endState, ExprNode *_cond, set<Assignment*> *_assign, ProbNode *_prob):
00097             shared(_shared), patternName(_patternName), matchName(_matchName),
00098             startState(_startState), endState(_endState), condition(_cond), assignments(_assign),
      probability(_prob) {};
00099 };
00100
00102 class LocalStateTemplate { // [YK]: <-- a class implementing Location
00103    public:
00105       string name;
00106
00108       set<TransitionTemplate*> transitions;  // [YK]: <-- adj. list of edges connecting locations
00109 };
00110
00111 /* Klasa reprezentująca pojedynczego agenta po wczytaniu jego opisu z pliku */
00112
00114 class AgentTemplate {
00115       TransitionTemplate transitionCache = TransitionTemplate(0, "", "", "", "", new ExprConst(1), new
      set<Assignment*>(), new ProbConst(1));
00116       float probabilityCache = 0.0;
00117       // identyfikator agenta
00118
00120       string ident; // [YK]: <-- Agent Type/Template Name (e.g., Voter, Train)
00121
00122       // stan startowy
00123
00125       string initState;
00126
00127       // zbiór zmiennych lokalnych (local)
00128
00130       set<string>* localVars;
00131
00132       // zbiór zmiennych trwałych (persistent)
00133
00135       set<string>* persistentVars;
00136
00137       // początkowa inicjacja
00138
00140       set<Assignment*>* initialAssignments;
00141
00142       // zbiór tranzycji
00143
00145       set<TransitionTemplate*>* transitions;
00146
00147       // mapa stanów lokalnych potrzebna do wygenerowania modelu
00148
00150       map<string,LocalStateTemplate*> localStateTemplates;  // [YK]: maps location name to location
      obj
00151
00152       // metoda wyznaczająca węzeł kolejny do danego, zależnie od tranzycji
00153
00154       virtual LocalState* genNextState(LocalState *state, TransitionTemplate *trans);
00155
00156    public:
00157       AgentTemplate();
00158
```

```
00159       // ustaw identyfikator agenta
00160
00164       virtual AgentTemplate& setIdent(string _ident);             // [YK]: <-- sets template Name
00165
00166       // ustaw identyfikator agenta
00167
00171       virtual AgentTemplate& setInitState(string _startState);    // [YK]: <-- sets initial location
00172
00173       // dodaj zmienną/zmienne lokalne
00174
00178       virtual AgentTemplate& addLocal(set<string> *variables);
00179
00180       // dodaj zmienne trwałe
00181
00185       virtual AgentTemplate& addPersistent(set<string> *variables);
00186
00187       // dodaj początkowe inicjacje
00188
00192       virtual AgentTemplate& addInitial(set<Assignment*> *assigns);
00193
00194       // dodaj tranzycję
00195
00199       virtual AgentTemplate& addTransition(TransitionTemplate *_transition);
00200
00201       // wygeneruj agenta do modelu
00202
00206       virtual Agent* generateAgent(int id) ;
00207
00208       friend class DotGraph;
00209 };
00210
00211 #endif
```

6.38 parser.h

```
00001 /* A Bison parser, made by GNU Bison 3.8.2.  */
00002
00003 /* Bison interface for Yacc-like parsers in C
00004
00005    Copyright (C) 1984, 1989-1990, 2000-2015, 2018-2021 Free Software Foundation,
00006    Inc.
00007
00008    This program is free software: you can redistribute it and/or modify
00009    it under the terms of the GNU General Public License as published by
00010    the Free Software Foundation, either version 3 of the License, or
00011    (at your option) any later version.
00012
00013    This program is distributed in the hope that it will be useful,
00014    but WITHOUT ANY WARRANTY; without even the implied warranty of
00015    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
00016    GNU General Public License for more details.
00017
00018    You should have received a copy of the GNU General Public License
00019    along with this program.  If not, see <https://www.gnu.org/licenses/>.  */
00020
00021 /* As a special exception, you may create a larger work that contains
00022    part or all of the Bison parser skeleton and distribute that work
00023    under terms of your choice, so long as that work isn't itself a
00024    parser generator using the skeleton or a modified version thereof
00025    as a parser skeleton.  Alternatively, if you modify or redistribute
00026    the parser skeleton itself, you may (at your option) remove this
00027    special exception, which will cause the skeleton and the resulting
00028    Bison output files to be licensed under the GNU General Public
00029    License without this special exception.
00030
00031    This special exception was added by the Free Software Foundation in
00032    version 2.2 of Bison.  */
00033
00034 /* DO NOT RELY ON FEATURES THAT ARE NOT DOCUMENTED in the manual,
00035    especially those whose name start with YY_ or yy_.  They are
00036    private implementation details that can be changed or removed.  */
00037
00038 #ifndef YY_YY_SRC_READER_PARSER_H_INCLUDED
00039 # define YY_YY_SRC_READER_PARSER_H_INCLUDED
00040 /* Debug traces.  */
00041 #ifndef YYDEBUG
00042 # define YYDEBUG 0
00043 #endif
00044 #if YYDEBUG
00045 extern int yydebug;
00046 #endif
00047
00048 /* Token kinds.  */
```

```
00049 #ifndef YYTOKENTYPE
00050 # define YYTOKENTYPE
00051   enum yytokentype
00052   {
00053     YYEMPTY = -2,
00054     YYEOF = 0,                     /* "end of file"  */
00055     YYerror = 256,                 /* error  */
00056     YYUNDEF = 257,                 /* "invalid token"  */
00057     T_AGENT = 258,                 /* T_AGENT  */
00058     T_INIT = 259,                  /* T_INIT  */
00059     T_LOCAL = 260,                 /* T_LOCAL  */
00060     T_INITIAL = 261,               /* T_INITIAL  */
00061     T_FORMULA = 262,               /* T_FORMULA  */
00062     T_PERSISTENT = 263,            /* T_PERSISTENT  */
00063     T_TO = 264,                    /* T_TO  */
00064     T_ASSIGN = 265,                /* T_ASSIGN  */
00065     T_OR = 266,                    /* T_OR  */
00066     T_AND = 267,                   /* T_AND  */
00067     T_EQ = 268,                    /* T_EQ  */
00068     T_NE = 269,                    /* T_NE  */
00069     T_GT = 270,                    /* T_GT  */
00070     T_GE = 271,                    /* T_GE  */
00071     T_LT = 272,                    /* T_LT  */
00072     T_LE = 273,                    /* T_LE  */
00073     T_NUM = 274,                   /* T_NUM  */
00074     T_SHARED = 275,                /* T_SHARED  */
00075     T_FLOAT = 276,                 /* T_FLOAT  */
00076     T_IDENT = 277,                 /* T_IDENT  */
00077     T_KNOW = 278,                  /* T_KNOW  */
00078     T_HARTLEY = 279,               /* T_HARTLEY  */
00079     T_PROB = 280,                  /* T_PROB  */
00080     T_REST = 281                   /* T_REST  */
00081   };
00082   typedef enum yytokentype yytoken_kind_t;
00083 #endif
00084
00085 /* Value type.  */
00086 #if ! defined YYSTYPE && ! defined YYSTYPE_IS_DECLARED
00087 union YYSTYPE
00088 {
00089 #line 26 "../src/reader/parser.y"
00090
00091        set<class AgentTemplate*>*  model;
00092        class ExprNode*             expr;
00093        class ProbNode*             prob;
00094        class Assignment*           assign;
00095        class TransitionTemplate*   trans;
00096        class AgentTemplate*        agent;
00097        set<class Assignment*>*     assignSet;
00098        char*                       ident;
00099        set<string>*                identSet;
00100        int                         val;
00101        float                       floatno;
00102        vector<class ExprNode*>*    condList;
00103
00104 #line 105 "../src/reader/parser.h"
00105
00106 };
00107 typedef union YYSTYPE YYSTYPE;
00108 # define YYSTYPE_IS_TRIVIAL 1
00109 # define YYSTYPE_IS_DECLARED 1
00110 #endif
00111
00112
00113 extern YYSTYPE yylval;
00114
00115
00116 int yyparse (void);
00117
00118
00119 #endif /* !YY_YY_SRC_READER_PARSER_H_INCLUDED  */
```

6.39 StrategyParser.cc File Reference

A strategy file parser. A parser for converting a text file into a strategy.

```
#include "StrategyParser.hpp"
#include "strategyReader/strategyNodes.hpp"
#include <stdio.h>
```

```
#include <string.h>
#include <tuple>
#include <iostream>
```

Functions

- int **stratparse** ()
- void **stratrestart** (FILE ∗)
- void **set_input_string_strat** (const char ∗in)

Variables

- StrategyCollectionTemplate **strategyCollectionTemplate**

6.39.1 Detailed Description

A strategy file parser. A parser for converting a text file into a strategy.

6.40 StrategyParser.hpp File Reference

Strategy file parser. A parser for converting a text file into a strategy.

```
#include "Types.hpp"
#include <tuple>
```

Classes

- class StrategyParser

  A parser for converting a text file into a strategy.

6.40.1 Detailed Description

Strategy file parser. A parser for converting a text file into a strategy.

6.41 StrategyParser.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef __STRATEGYPARSER_HPP
00008 #define __STRATEGYPARSER_HPP
00009
00010 #include "Types.hpp"
00011 #include <tuple>
00012
00013 using namespace std;
00014
00016 class StrategyParser {
00017     public:
00018         StrategyParser();
00019         ~StrategyParser();
00020         StrategyCollection* parse(string fileName);
00021
00022     protected:
00023         // @internal
00024 };
00025
00026 #endif
```

6.42 strategyNodes.hpp File Reference

Strategy parser templates. Class for setting up a new objects from a parser.

```
#include <string>
#include <set>
#include <map>
#include "../Types.hpp"
```

Classes

- class ActionTemplate

  Represents a selected action.
- class StrategyCollectionTemplate

6.42.1 Detailed Description

Strategy parser templates. Class for setting up a new objects from a parser.

6.43 strategyNodes.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef __STRATEGY_NODES_H
00008 #define __STRATEGY_NODES_H
00009
00010 #include <string>
00011 #include <set>
00012 #include <map>
00013
00014 #include "../Types.hpp"
00015
00016 using namespace std;
00017
00019 class ActionTemplate {
00020    public:
00024       ActionTemplate(vector<string>* _states, string _actionName);
00026       vector<string> *states;
00028       string hash;
00030       string actionName;
00031 };
00032
00034 class StrategyCollectionTemplate {
00035    public:
00037       StrategyCollectionTemplate();
00039       map<string, ActionTemplate> selectedStrategy;
00042       void addAction(vector<string>* _states, string _actionName);
00043 };
00044
00045 #endif
```

6.44 strategyParser.h

```
00001 /* A Bison parser, made by GNU Bison 3.8.2.  */
00002
00003 /* Bison interface for Yacc-like parsers in C
00004
00005    Copyright (C) 1984, 1989-1990, 2000-2015, 2018-2021 Free Software Foundation,
00006    Inc.
00007
00008    This program is free software: you can redistribute it and/or modify
00009    it under the terms of the GNU General Public License as published by
00010    the Free Software Foundation, either version 3 of the License, or
00011    (at your option) any later version.
00012
00013    This program is distributed in the hope that it will be useful,
00014    but WITHOUT ANY WARRANTY; without even the implied warranty of
00015    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
00016    GNU General Public License for more details.
00017
00018    You should have received a copy of the GNU General Public License
00019    along with this program.  If not, see <https://www.gnu.org/licenses/>.  */
00020
00021 /* As a special exception, you may create a larger work that contains
00022    part or all of the Bison parser skeleton and distribute that work
00023    under terms of your choice, so long as that work isn't itself a
00024    parser generator using the skeleton or a modified version thereof
00025    as a parser skeleton.  Alternatively, if you modify or redistribute
00026    the parser skeleton itself, you may (at your option) remove this
00027    special exception, which will cause the skeleton and the resulting
00028    Bison output files to be licensed under the GNU General Public
00029    License without this special exception.
00030
00031    This special exception was added by the Free Software Foundation in
00032    version 2.2 of Bison.  */
00033
00034 /* DO NOT RELY ON FEATURES THAT ARE NOT DOCUMENTED in the manual,
00035    especially those whose name start with YY_ or yy_.  They are
00036    private implementation details that can be changed or removed.  */
00037
00038 #ifndef YY_STRAT_SRC_STRATEGYREADER_STRATEGYPARSER_H_INCLUDED
00039 # define YY_STRAT_SRC_STRATEGYREADER_STRATEGYPARSER_H_INCLUDED
00040 /* Debug traces.  */
00041 #ifndef STRATDEBUG
00042 # if defined YYDEBUG
00043 #if YYDEBUG
00044 #   define STRATDEBUG 1
00045 #  else
00046 #   define STRATDEBUG 0
00047 #  endif
00048 # else /* ! defined YYDEBUG */
00049 #  define STRATDEBUG 0
00050 # endif /* ! defined YYDEBUG */
00051 #endif  /* ! defined STRATDEBUG */
00052 #if STRATDEBUG
00053 extern int stratdebug;
00054 #endif
00055
00056 /* Token kinds.  */
00057 #ifndef STRATTOKENTYPE
00058 # define STRATTOKENTYPE
00059   enum strattokentype
00060   {
00061     STRATEMPTY = -2,
00062     STRATEOF = 0,                  /* "end of file"  */
00063     STRATerror = 256,              /* error  */
00064     STRATUNDEF = 257,              /* "invalid token"  */
00065     T_TO = 258,                    /* T_TO  */
00066     T_ASSIGN = 259,                /* T_ASSIGN  */
00067     T_OR = 260,                    /* T_OR  */
00068     T_AND = 261,                   /* T_AND  */
00069     T_EQ = 262,                    /* T_EQ  */
00070     T_NE = 263,                    /* T_NE  */
00071     T_GT = 264,                    /* T_GT  */
00072     T_GE = 265,                    /* T_GE  */
00073     T_LT = 266,                    /* T_LT  */
00074     T_LE = 267,                    /* T_LE  */
00075     T_NUM = 268,                   /* T_NUM  */
00076     T_SHARED = 269,                /* T_SHARED  */
00077     T_FLOAT = 270,                 /* T_FLOAT  */
00078     T_IDENT = 271                  /* T_IDENT  */
00079   };
00080   typedef enum strattokentype strattoken_kind_t;
00081 #endif
00082
00083 /* Value type.  */
00084 #if ! defined STRATSTYPE && ! defined STRATSTYPE_IS_DECLARED
00085 union STRATSTYPE
```

```
00086 {
00087 #line 26 "../src/strategyReader/strategyParser.y"
00088
00089       char*                       expr;
00090       int                         val;
00091       float                       floatno;
00092       char*                       ident;
00093       vector<string>*             identList;
00094
00095 #line 96 "../src/strategyReader/strategyParser.h"
00096
00097 };
00098 typedef union STRATSTYPE STRATSTYPE;
00099 # define STRATSTYPE_IS_TRIVIAL 1
00100 # define STRATSTYPE_IS_DECLARED 1
00101 #endif
00102
00103
00104 extern STRATSTYPE stratlval;
00105
00106
00107 int stratparse (void);
00108
00109
00110 #endif /* !YY_STRAT_SRC_STRATEGYREADER_STRATEGYPARSER_H_INCLUDED  */
```

6.45 Types.hpp File Reference

Custom data structures. Data structures and classes containing model data.

```
#include <map>
#include <set>
#include <stack>
#include <queue>
#include <string>
#include <utility>
#include <vector>
#include "LocalState.hpp"
#include "LocalTransition.hpp"
#include "GlobalState.hpp"
#include "GlobalTransition.hpp"
#include "GlobalModel.hpp"
#include "EpistemicClass.hpp"
#include "Agent.hpp"
#include "reader/expressions.hpp"
#include <atomic>
#include "TypesDependency.hpp"
```

Classes

- struct Var

  Represents a variable in the model, containing name, initial value and persistence.
- struct Condition

  Represents a condition for LocalTransition.
- struct FormulaTemplate

  Contains a template for coalition of Agent as string from the formula.
- struct Formula

  Contains the processed verification formula.
- struct LocalModels

  Represents a single local model, contains all agents and variables.
- struct Result

Enumerations

- enum ProbabilitySign { NONE , EQ , NE , GT , GE , LT , LE }

  Probability inequality sign used for the probabilistic verification.
- enum **VerificationFormulaMode** { **NOT_SET** = 0 , **F** = 1 , **G** = 2 }

6.45.1 Detailed Description

Custom data structures. Data structures and classes containing model data.

6.45.2 Enumeration Type Documentation

6.45.2.1 ProbabilitySign

```
enum ProbabilitySign
```

Probability inequality sign used for the probabilistic verification.

Enumerator

NONE	No probability detected.
EQ	Equal (==)
NE	Not equal (!=)
GT	Greater than (>)
GE	Greater equal (>=)
LT	Less than (<)
LE	Less equal (<=)

6.46 Types.hpp

Go to the documentation of this file.

```
00001
00007 #ifndef SELENE_TYPES
00008 #define SELENE_TYPES
00009
00010 #include <map>
00011 #include <set>
00012
00013 #include <stack>
00014 #include <queue>
00015 #include <string>
00016 #include <utility>
00017 #include <vector>
00018
00019 #include "LocalState.hpp"
00020 #include "LocalTransition.hpp"
00021
00022 #include "GlobalState.hpp"
00023 #include "GlobalTransition.hpp"
00024 #include "GlobalModel.hpp"
00025
00026 #include "EpistemicClass.hpp"
00027 #include "Agent.hpp"
```

```
00028
00029 #include "reader/expressions.hpp"
00030
00031 #include <atomic> //std::atomic_uint32_t
00032
00033 #include "TypesDependency.hpp"
00034
00035 using namespace std;
00036
00038 enum ProbabilitySign {
00039     NONE,
00040     EQ,
00041     NE,
00042     GT,
00043     GE,
00044     LT,
00045     LE
00046 };
00047
00049 struct Var {
00051     string name;
00052
00054     int initialValue;
00055
00057     bool persistent;
00058
00060     Agent *agent;
00061 };
00062
00064 struct Condition {
00066     Var* var;
00067
00069     ConditionOperator conditionOperator;
00070
00072     int comparedValue;
00073 };
00074
00076 struct FormulaTemplate {
00077     set<string>* coalition; // this will be replaced by an Agent pointer upon instantiation of a
      formula
00078     vector<ExprNode*>* formula;
00079     bool isF;
00080     ProbNode* probability;
00081     string probabilitySign = "";
00082 };
00083
00085 struct Formula {
00087     set<Agent*> coalition;
00088     vector<ExprNode*>* p; // [YK]: temporary solution to encode «coalution» G p
00089     bool isF;
00090     bool isCTL;
00091     float probability = 1;
00092     ProbabilitySign probabilitySign = ProbabilitySign::NONE;
00093 };
00094
00096 struct LocalModels {
00098     vector<Agent*> agents;
00099     // agents[i].id == i
00100 };
00101
00102
00103 struct Result {
00104     bool verificationResult = false;
00105     ProbabilityTrueFalse probabilityResult;
00106 };
00107
00108 enum VerificationFormulaMode {
00109     NOT_SET = 0,
00110     F = 1,
00111     G = 2
00112 };
00113
00114 #endif // SELENE_TYPES
```

6.47 TypesDependency.hpp File Reference

Custom data structures, using other custom data structures. Data structures and classes containing model data that are using other custom data structures.

```
#include <bitset>
#include <queue>
```

Classes

- struct StrategyEntry
- struct StrategyBitsComparator

  A comparator for two bitsets containing values for the global states.
- struct ProbabilityTrueFalse

  Contains probability of a state being in a true or false state.
- class ProbabilityEntry

  Class for handling the probability operations during probability verification.
- struct StateVerificationInfo

  Handles all single state verification information during probabilistic verification.
- struct Action

  Single action template for probabilistic verification.
- class StrategyCollection

  Handles currently selected strategy when the strategy is set.

Macros

- #define **STRATEGY_BITS** 64

Enumerations

- enum **VerifResult** { **NOT_VERIFIED** = 0 , **TRUE** = 1 , **FALSE** = 2 }
- enum StrategyEntryType { NOT_MODIFIED , ADDED }

  StrategyEntry entry type.
- enum HistoryEntryType {
  DECISION , STATE_STATUS , CONTEXT , MARK_DECISION_AS_INVALID ,
  UNCONTROLLED_DECISION , PROBABILITY , ANSWER_PROBABILITY }

  HistoryEntry entry type.
- enum ProbabilityCalculationType { **SUM_PROBABILITY** , **MIN_PROBABILITY** , **NONE_PROBABILITY** }

  Contains the type of the probability calculation in a given node.

6.47.1 Detailed Description

Custom data structures, using other custom data structures. Data structures and classes containing model data that are using other custom data structures.

6.47.2 Enumeration Type Documentation

6.47.2.1 HistoryEntryType

```
enum HistoryEntryType
```

HistoryEntry entry type.

Enumerator

DECISION	Made the decision to go to a state using a transition.
STATE_STATUS	Changed verification status.
CONTEXT	Recursion has gone deeper.
MARK_DECISION_AS_INVALID	Marking a transition as invalid.
UNCONTROLLED_DECISION	One uncontrolled choice from a set, from which all of them has to be OK.
PROBABILITY	There's a change in probability.
ANSWER_PROBABILITY	There's a change in the answer probability.

6.47.2.2 StrategyEntryType

```
enum StrategyEntryType
```

StrategyEntry entry type.

Enumerator

NOT_MODIFIED	Entry didn't have to get modified.
ADDED	Entry got added to the map.

6.48 TypesDependency.hpp

Go to the documentation of this file.

```
00001
00007 #define STRATEGY_BITS 64
00008
00009 #ifndef TYPES_DEPENDENCY
00010 #define TYPES_DEPENDENCY
00011
00012 #include <bitset>
00013 #include <queue>
00014
00015 using namespace std;
00016
00017 enum VerifResult {
00018     NOT_VERIFIED = 0,
00019     TRUE = 1,
00020     FALSE = 2
00021 };
00022
00024 enum StrategyEntryType {
00025     NOT_MODIFIED,
00026     ADDED,
00027 };
00028
00029 struct StrategyEntry {
00030     StrategyEntryType type = NOT_MODIFIED;
00031     bitset<STRATEGY_BITS> globalValues = 0;
00032     string* actionName;
00033 };
00034
00036 struct StrategyBitsComparator {
00037     bool operator() (const bitset<STRATEGY_BITS> &b1, const bitset<STRATEGY_BITS> &b2) const {
00038         return b1.to_ulong() < b2.to_ulong();
00039     }
00040 };
00041
00043 enum HistoryEntryType {
00044     DECISION,
00045     STATE_STATUS,
00046     CONTEXT,
```

```
00047     MARK_DECISION_AS_INVALID,
00048     UNCONTROLLED_DECISION,
00049     PROBABILITY,
00050     ANSWER_PROBABILITY
00051 };
00052
00054 enum ProbabilityCalculationType {
00055     SUM_PROBABILITY,
00056     MIN_PROBABILITY,
00057     NONE_PROBABILITY
00058 };
00059
00061 struct ProbabilityTrueFalse {
00062     float probabilityTrue = 0.0;
00063     float probabilityFalse = 0.0;
00064 };
00065
00067 class ProbabilityEntry {
00068     public:
00069         ProbabilityEntry() {} ;
00070         ProbabilityEntry(ProbabilityCalculationType mode) { probabilityCalculationType = mode; };
00071         ~ProbabilityEntry() {};
00072         void changeVerificationMode(ProbabilityCalculationType mode) { probabilityCalculationType =
      mode; }
00073         ProbabilityCalculationType getVerificationMode() { return probabilityCalculationType; }
00074         void changeSumProbabilityTrue(float changeProbabilityBy) { sumProbability.probabilityTrue +=
      changeProbabilityBy; }
00075         void changeSumProbabilityFalse(float changeProbabilityBy) { sumProbability.probabilityFalse +=
      changeProbabilityBy; }
00076         ProbabilityTrueFalse getSumProbability() { return sumProbability; }
00077         void setSumProbability(ProbabilityTrueFalse newSumProbability ) { sumProbability =
      newSumProbability; }
00078         void changeMinProbabilityTrue(float newProbability) { (minProbability.probabilityTrue >
      newProbability ? minProbability.probabilityTrue = newProbability : 0); }
00079         void changeMinProbabilityFalse(float newProbability) { (minProbability.probabilityFalse <
      newProbability ? minProbability.probabilityFalse = newProbability : 0); }
00080         ProbabilityTrueFalse getMinProbability() { return minProbability; }
00081         void setMinProbability(ProbabilityTrueFalse newMinProbability ) { minProbability =
      newMinProbability; }
00082         ProbabilityTrueFalse returnProbability() { return (probabilityCalculationType ==
      ProbabilityCalculationType::MIN_PROBABILITY ? minProbability : sumProbability); }
00083     private:
00084         ProbabilityCalculationType probabilityCalculationType =
      ProbabilityCalculationType::SUM_PROBABILITY;
00085         ProbabilityTrueFalse sumProbability = {0.0, 0.0};
00086         ProbabilityTrueFalse minProbability = {1.0, 0.0};
00087 };
00088
00090 struct StateVerificationInfo {
00091     GlobalState* globalState = nullptr;
00092     StateVerificationInfo* fromState = nullptr;
00093     queue<GlobalTransition*> controlledTransitionsLeftToProcess;
00094     queue<GlobalTransition*> uncontrolledTransitionsLeftToProcess;
00095     map<string, set<GlobalTransition*>> controlledProbabilisticTransitionsLeftToProcess;
00096     map<string, set<GlobalTransition*>> uncontrolledProbabilisticTransitionsLeftToProcess;
00097     int depth = 0;
00098     bool processed = false;
00099     VerifResult verifResult = VerifResult::NOT_VERIFIED;
00100     bool controlled = false;
00101     bool uncontrolled = false;
00102     bool hasControlledProbabilistic = false;
00103     bool hasUncontrolledProbabilistic = false;
00104     bool hasValidControlledTransition = false;
00105     bool hasValidUncontrolledTransition = true;
00106     bool isControlledByCoalition = false;
00107     bool gotThroughPreselectedTransition = false;
00108     bool gotResponseFromOtherState = false;
00109 };
00110
00112 struct Action {
00114     vector<string> *states;
00116     string hash;
00118     string actionName;
00119 };
00120
00122 class StrategyCollection {
00123     private:
00124         map<string, Action> selectedStrategy;
00125     public:
00126         StrategyCollection() {} ;
00127         ~StrategyCollection() {};
00128         StrategyCollection(string hash, Action action) { selectedStrategy = {{hash, action}}; };
00129         void addAction(Action action) {
00130             selectedStrategy[action.hash] = action;
00131         };
00132         void addStrategy(StrategyCollection strategy) {
00133             for (auto item : strategy.getStrategy()) {
```

```
00134                 selectedStrategy[item.first] = item.second;
00135             }
00136         }
00137         void clear() {
00138             selectedStrategy.clear();
00139         };
00140         map<string, Action> getStrategy() {
00141             return selectedStrategy;
00142         };
00143 };
00144
00145 #endif // TYPES_DEPENDENCY
```

6.49 Utils.cpp File Reference

Utility functions. A collection of utility functions to use in the project.

```
#include "Utils.hpp"
#include <fstream>
#include "GlobalModelGenerator.hpp"
#include <map>
#include <algorithm>
#include <iostream>
#include <cstdio>
```

Functions

- string envToString (map< string, int > env)

  Converts a map of string and int to a string.
- string agentToString (Agent *agt)

  Converts pointer to an Agant into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.
- string localModelsToString (LocalModels *lm)

  Converts pointer to the LocalModels into a string cointaining all Agent instances from the model, initial values of the variables and names of the persistent values.
- void outputGlobalModel (GlobalModel *globalModel)

  Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.
- unsigned long **getMemCap** ()
- void **loadConfigFromFile** (string filename)
- void **loadConfigFromArgs** (int argc, char **argv)
- void tarjanVisit (LocalState *v, map< int, int > *dindex, map< int, int > *lowlink, stack< LocalState * > *stack, map< int, bool > *onstack, int *depth, vector< set< LocalState * > > *comp)

  Utility function for SCC-computatation.
- vector< set< LocalState * > > getLocalStatesSCC (Agent *agt)

  a quick implementation of a Tarjan SCC algorithm (based on DFS)
- map< LocalState *, vector< GlobalState * > > **getContextModel** (Formula *formula, LocalModels *localModels, Agent *agt)

Variables

- Cfg **config**

6.49.1 Detailed Description

Utility functions. A collection of utility functions to use in the project.

6.49.2 Function Documentation

6.49.2.1 agentToString()

```
string agentToString (
            Agent * agt )
```

Converts pointer to an Agant into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.

Parameters

agt	Pointer to an Agent to parse into a string.

Returns

String containing all of Agent data.

6.49.2.2 envToString()

```
string envToString (
            map< string, int > env )
```

Converts a map of string and int to a string.

Parameters

env	Map to be converted into a string.

Returns

Returns string " (first_name, second_name, ..., last_name=int_value)"

6.49.2.3 getLocalStatesSCC()

```
vector< set< LocalState * > > getLocalStatesSCC (
            Agent * agt )
```

a quick implementation of a Tarjan SCC algorithm (based on DFS)

Parameters

agt	- an agent whose local graph will be inspected

Returns

localStates partition in a form of the vector, where each set correponds to a SCC

6.49.2.4 localModelsToString()

```
string localModelsToString (
            LocalModels * lm )
```

Converts pointer to the LocalModels into a string cointaining all Agent instances from the model, initial values of the variables and names of the persistent values.

Parameters

lm	Pointer to the local model to parse into a string.

Returns

String containing all of LocalModels data.

6.49.2.5 outputGlobalModel()

```
void outputGlobalModel (
            GlobalModel * globalModel )
```

Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.

Parameters

globalModel	Pointer to a GlobalModel to print into the console.

6.49.2.6 tarjanVisit()

```
void tarjanVisit (
            LocalState * v,
            map< int, int > * dindex,
            map< int, int > * lowlink,
            stack< LocalState * > * stack,
            map< int, bool > * onstack,
            int * depth,
            vector< set< LocalState * > > * comp )
```

Utility function for SCC-computatation.

Parameters

v	- current vertex
dindex	- vertex.index (alt. vertex.num)

Parameters

lowlink	- vertex.lowlink (by df. lowest dindex in the same scc reachable from vertex using tree edges followed by at most one back/cross edge)
stack	- holds candidates for SCC
onstack	- used as condition for back/cross-edge case
depth	- next available (discovery) index
comp	- result of SCC partitioning

6.50 Utils.hpp File Reference

```
#include "Types.hpp"
#include "Constants.hpp"
#include <map>
#include <string>
#include <unistd.h>
#include <sys/time.h>
#include <iostream>
#include <fstream>
```

Functions

- string envToString (map< string, int > env)

  Converts a map of string and int to a string.
- string agentToString (Agent *agt)

  Converts pointer to an Agant into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.
- string localModelsToString (LocalModels *lm)

  Converts pointer to the LocalModels into a string cointaining all Agent instances from the model, initial values of the variables and names of the persistent values.
- void outputGlobalModel (GlobalModel *globalModel)

  Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.
- unsigned long **getMemCap** ()
- vector< set< LocalState * > > getLocalStatesSCC (Agent *agt)

  a quick implementation of a Tarjan SCC algorithm (based on DFS)
- map< LocalState *, vector< GlobalState * > > **getContextModel** (Formula *formula, LocalModels *localModels, Agent *agt)
- void **loadConfigFromFile** (string filename="config.txt")
- void **loadConfigFromArgs** (int argc, char **argv)

6.50.1 Function Documentation

6.50.1.1 agentToString()

```
string agentToString (
            Agent * agt )
```

Converts pointer to an Agant into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.

Parameters

agt	Pointer to an Agent to parse into a string.

Returns

String containing all of Agent data.

6.50.1.2 envToString()

```
string envToString (
            map< string, int > env )
```

Converts a map of string and int to a string.

Parameters

env	Map to be converted into a string.

Returns

Returns string " (first_name, second_name, ..., last_name=int_value)"

6.50.1.3 getLocalStatesSCC()

```
vector< set< LocalState * > > getLocalStatesSCC (
            Agent * agt )
```

a quick implementation of a Tarjan SCC algorithm (based on DFS)

Parameters

agt	- an agent whose local graph will be inspected

Returns

localStates partition in a form of the vector, where each set correponds to a SCC

6.50.1.4 localModelsToString()

```
string localModelsToString (
            LocalModels * lm )
```

Converts pointer to the LocalModels into a string cointaining all Agent instances from the model, initial values of the variables and names of the persistent values.

Parameters

lm	Pointer to the local model to parse into a string.

Returns

String containing all of LocalModels data.

6.50.1.5 outputGlobalModel()

```
void outputGlobalModel (
            GlobalModel * globalModel )
```

Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.

Parameters

globalModel	Pointer to a GlobalModel to print into the console.

6.51 Utils.hpp

Go to the documentation of this file.

```
00001
00005 #ifndef STV_TYPES
00006 #define STV_TYPES
00007
00008 #include "Types.hpp"
00009 #include "Constants.hpp"
00010 #include <map>
00011 #include <string>
00012 #include <unistd.h>
00013 #include <sys/time.h>
00014 #include <iostream>
00015 #include <fstream>
00016
00017 using namespace std;
00018
00019 string envToString(map<string, int> env);
00020 string agentToString(Agent* agt);
00021 string localModelsToString(LocalModels* lm);
00022 void outputGlobalModel(GlobalModel* globalModel);
00023 unsigned long getMemCap();
00024 vector<set<LocalState*>> getLocalStatesSCC(Agent* agt);
00025 map<LocalState*,vector<GlobalState*>> getContextModel(Formula* formula, LocalModels* localModels,
      Agent* agt);
00026 void loadConfigFromFile(string filename="config.txt");
00027 void loadConfigFromArgs(int argc, char** argv);
00028
00029 #endif // STV_TYPES
```

6.52 Verification.cpp File Reference

Class for verification of the formula on a model. Class for verification of the specified formula on a specified model.

```
#include "Verification.hpp"
#include <bits/stdc++.h>
```

Macros

- #define **DEPTH_PREFIX** string(depth ∗ 4, ' ')

Functions

- string verStatusToStr (GlobalStateVerificationStatus status)

  Converts global verification status into a string.
- void dbgVerifStatus (string prefix, GlobalState ∗gs, GlobalStateVerificationStatus st, string reason)

  Print a debug message of a verification status to the console.
- void dbgHistEnt (string prefix, HistoryEntry ∗h)

  Print a single debug message with a history entry to the console.

Variables

- Cfg **config**

6.52.1 Detailed Description

Class for verification of the formula on a model. Class for verification of the specified formula on a specified model.

6.52.2 Function Documentation

6.52.2.1 dbgHistEnt()

```
void dbgHistEnt (
            string prefix,
            HistoryEntry * h )
```

Print a single debug message with a history entry to the console.

Parameters

prefix	A prefix string to append to the front of the entry.
h	A pointer to the HistoryEntry struct which will be printed out.

6.52.2.2 dbgVerifStatus()

```
void dbgVerifStatus (
            string prefix,
            GlobalState * gs,
            GlobalStateVerificationStatus st,
            string reason )
```

Print a debug message of a verification status to the console.

Parameters

prefix	A prefix string to append to the front of every entry.
gs	Pointer to a GlobalState.
st	Enum with a verification status of a global state.
reason	String with a reason why the function was called, e.g. "entered state", "all passed".

6.52.2.3 verStatusToStr()

```
string verStatusToStr (
            GlobalStateVerificationStatus status )
```

Converts global verification status into a string.

Parameters

status	Enum value to be converted.

Returns

Verification status converted into a string.

6.53 Verification.hpp File Reference

```
#include <stack>
#include "Types.hpp"
#include "TypesDependency.hpp"
#include "GlobalModelGenerator.hpp"
#include <bitset>
```

Classes

- struct HistoryEntry

  Structure used to save model traversal history.
- class HistoryDbg

  Stores history and allows displaying it to the console.
- class Verification

  A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

Macros

- #define **STRATEGY_BITS** 64

Enumerations

- enum TraversalMode { NORMAL , REVERT , RESTORE }

  Current model traversal mode.

Functions

- string verStatusToStr (GlobalStateVerificationStatus status)

  Converts global verification status into a string.

6.53.1 Enumeration Type Documentation

6.53.1.1 TraversalMode

```
enum TraversalMode
```

Current model traversal mode.

Enumerator

NORMAL	Normal model traversal.
REVERT	Backtracking through recursion with state rollback.
RESTORE	Backtracking through recursion.

6.53.2 Function Documentation

6.53.2.1 verStatusToStr()

```
string verStatusToStr (
            GlobalStateVerificationStatus status )
```

Converts global verification status into a string.

Parameters

status	Enum value to be converted.

Returns

Verification status converted into a string.

6.54 Verification.hpp

Go to the documentation of this file.

```
00001
00004 #define STRATEGY_BITS 64
00005
00006 #ifndef SELENE_VERIFICATION
00007 #define SELENE_VERIFICATION
00008
00009 #include <stack>
00010 #include "Types.hpp"
00011 #include "TypesDependency.hpp"
00012 #include "GlobalModelGenerator.hpp"
00013 #include <bitset>
00014
```

```
00015 string verStatusToStr(GlobalStateVerificationStatus status);
00016
00018 struct HistoryEntry {
00020     HistoryEntryType type;
00022     GlobalState* globalState;
00024     GlobalTransition* decision;
00026     bool globalTransitionControlled;
00028     GlobalStateVerificationStatus prevStatus;
00030     GlobalStateVerificationStatus newStatus;
00032     int depth;
00034     StrategyEntry strategy;
00036     HistoryEntry* prev;
00038     HistoryEntry* next;
00041     string toString() {
00042         char buff[1024] = { 0 };
00043         if (this->type == HistoryEntryType::DECISION) {
00044             snprintf(buff, sizeof(buff), "decision in %s: to %s", this->globalState->hash.c_str(),
      this->decision->to->hash.c_str());
00045         }
00046         else if (this->type == HistoryEntryType::STATE_STATUS) {
00047             snprintf(buff, sizeof(buff), "stateVerifStatus of %s: %s -> %s",
      this->globalState->hash.c_str(), verStatusToStr(this->prevStatus).c_str(),
      verStatusToStr(this->newStatus).c_str());
00048         }
00049         else if (this->type == HistoryEntryType::CONTEXT) {
00050             snprintf(buff, sizeof(buff), "context in %s at depth %i: to %s (%s)",
      this->globalState->hash.c_str(), this->depth, this->decision->to->hash.c_str(),
      this->globalTransitionControlled ? "controlled" : "uncontrolled");
00051         }
00052         else if (this->type == HistoryEntryType::MARK_DECISION_AS_INVALID) {
00053             snprintf(buff, sizeof(buff), "markInvalid in %s: to %s", this->globalState->hash.c_str(),
      this->decision->to->hash.c_str());
00054         }
00055         else if (this->type == HistoryEntryType::UNCONTROLLED_DECISION) {
00056             snprintf(buff, sizeof(buff), "uncontrolledDecision in %s at depth %i: to %s (%s)",
      this->globalState->hash.c_str(), this->depth, this->decision->to->hash.c_str(),
      this->globalTransitionControlled ? "controlled" : "uncontrolled");
00057         }
00058         return string(buff);
00059     };
00060 };
00061
00063 class HistoryDbg {
00064 public:
00066     vector<pair<HistoryEntry*, char>> entries;
00067     HistoryDbg();
00068     ~HistoryDbg();
00069     void addEntry(HistoryEntry* entry);
00070     void markEntry(HistoryEntry* entry, char chr);
00071     void print(string prefix);
00072     HistoryEntry* cloneEntry(HistoryEntry* entry);
00073 };
00074
00075 // On-the-fly traversal mode
00077 enum TraversalMode {
00078     NORMAL,
00079     REVERT,
00080     RESTORE,
00081 };
00082
00084 class Verification {
00085 public:
00086     Verification(GlobalModelGenerator* generator);
00087     ~Verification();
00088     bool verify();
00089     bool fixpointVerify();
00090     void historyDecisionsERR();
00091     map<bitset<STRATEGY_BITS>, string, StrategyBitsComparator> getNaturalStrategy();
00092     vector<tuple<vector<tuple<bool, string>>, string>> getReducedStrategy();
00093     int getStrategyComplexity();
00094     Result verifyStrategy();
00095     Result verifyMDP();
00096 protected:
00098     TraversalMode mode;
00100     GlobalState* revertToGlobalState;
00102     stack<HistoryEntry*> historyToRestore;
00104     GlobalModelGenerator* generator;
00106     HistoryEntry* historyStart;
00108     HistoryEntry* historyEnd;
00110     map<bitset<STRATEGY_BITS>, string, StrategyBitsComparator> naturalStrategy;
00112     short strategyVariableLimit;
00114     vector<string> variableNames;
00116     int reductionComplexityBefore;
00117
00118     bool verifyLocalStates(vector<LocalState*>* localStates, GlobalState* globalState);
00119     bool verifyGlobalState(GlobalState* globalState, int depth);
00120     bool isGlobalTransitionControlledByCoalition(GlobalTransition* globalTransition);
```

```
00121     bool isAgentInCoalition(Agent* agent);
00122     EpistemicClass* getEpistemicClassForGlobalState(GlobalState* globalState);
00123     bool areGlobalStatesInTheSameEpistemicClass(GlobalState* globalState1, GlobalState* globalState2);
00124     void addHistoryDecision(GlobalState* globalState, GlobalTransition* decision);
00125     void addHistoryStateStatus(GlobalState* globalState, GlobalStateVerificationStatus prevStatus,
      GlobalStateVerificationStatus newStatus);
00126     bool addHistoryContext(GlobalState* globalState, int depth, GlobalTransition* decision, bool
      globalTransitionControlled);
00127     void addHistoryMarkDecisionAsInvalid(GlobalState* globalState, GlobalTransition* decision);
00128     void addHistoryUncontrolledDecision(GlobalState* globalState, GlobalTransition* decision);
00129     HistoryEntry* newHistoryMarkDecisionAsInvalid(GlobalState* globalState, GlobalTransition*
      decision);
00130     bool revertLastDecision(int depth);
00131     void undoLastHistoryEntry(bool freeMemory);
00132     void undoHistoryUntil(HistoryEntry* historyEntry, bool inclusive, int depth);
00133     void printCurrentHistory(int depth);
00134     bool equivalentGlobalTransitions(GlobalTransition* globalTransition1, GlobalTransition*
      globalTransition2);
00135     bool checkUncontrolledSet(set<GlobalTransition*> uncontrolledGlobalTransitions, GlobalState*
      globalState, int depth, bool hasOmittedTransitions, bool mixed = false);
00136     bool verifyTransitionSets(set<GlobalTransition*> controlledGlobalTransitions,
      set<GlobalTransition*> uncontrolledGlobalTransitions, GlobalState* globalState, int depth, bool
      hasOmittedTransitions, bool isFMode, bool mixed = false);
00137     bool restoreHistory(GlobalState* globalState, GlobalTransition* globalTransition, int depth, bool
      controlled);
00138     bool minFixpointVerify();
00139     bool maxFixpointVerify();
00140     bitset<STRATEGY_BITS> globalStateToValueBits(GlobalState* globalState);
00141     vector<tuple<vector<tuple<bool, string>, string>> reduceStrategy(vector<tuple<vector<tuple<bool,
      string>, string>> strategyEntries, short lockedColumn = 0, bool upperHalf = false);
00142     void increaseProbability(GlobalState* currentState, GlobalTransition* decision);
00143     void lowerProbability(GlobalState* currentStateFrom, GlobalState* currentStateTo,
      set<LocalTransition*> decision);
00144 };
00145
00146 #endif // SELENE_VERIFICATION
```

6.55 VerificationIterative.cpp File Reference

Class for iteratively generated verification of the formula on a model. Class for iteratively generated verification of the specified formula on a specified model.

```
#include "VerificationIterative.hpp"
#include <bits/stdc++.h>
```

Macros

- #define **DEPTH_PREFIX** string(depth ∗ 4, ' ')

Functions

- string verifStatusToStr (GlobalStateVerificationStatus status)

  Converts global verification status into a string.

Variables

- Cfg **config**

6.55.1 Detailed Description

Class for iteratively generated verification of the formula on a model. Class for iteratively generated verification of the specified formula on a specified model.

6.55.2 Function Documentation

6.55.2.1 verifStatusToStr()

```
string verifStatusToStr (
            GlobalStateVerificationStatus status )
```

Converts global verification status into a string.

Parameters

status	Enum value to be converted.

Returns

Verification status converted into a string.

6.56 VerificationIterative.hpp File Reference

```
#include <stack>
#include "Types.hpp"
#include "TypesDependency.hpp"
#include "GlobalModelGenerator.hpp"
#include <bitset>
```

Classes

- struct DecisionEntry
- class VerificationIterative

  A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

Enumerations

- enum GraphTraversalMode { NORMAL_TRAVERSAL , RESTORE_STATES }

  Current model traversal mode.

Functions

- string verifStatusToStr (GlobalStateVerificationStatus status)

  Converts global verification status into a string.

6.56.1 Enumeration Type Documentation

6.56.1.1 GraphTraversalMode

```
enum GraphTraversalMode
```

Current model traversal mode.

Enumerator

NORMAL_TRAVERSAL	Normal model traversal.
RESTORE_STATES	Reconstructing recursion.

6.56.2 Function Documentation

6.56.2.1 verifStatusToStr()

```
string verifStatusToStr (
            GlobalStateVerificationStatus status )
```

Converts global verification status into a string.

Parameters

status	Enum value to be converted.

Returns

Verification status converted into a string.

6.57 VerificationIterative.hpp

Go to the documentation of this file.

```
00001
00004 #ifndef VERIFICATION_ITERATIVE
00005 #define VERIFICATION_ITERATIVE
00006
00007 #include <stack>
00008 #include "Types.hpp"
00009 #include "TypesDependency.hpp"
00010 #include "GlobalModelGenerator.hpp"
00011 #include <bitset>
00012
00013 string verifStatusToStr(GlobalStateVerificationStatus status);
00014
00016 struct DecisionEntry {
00018     HistoryEntryType type = HistoryEntryType::CONTEXT;
00020     GlobalState* globalState = nullptr;
00022     GlobalTransition* decision = nullptr;
00024     bool globalTransitionControlled = false;
00026     GlobalStateVerificationStatus prevStatus = GlobalStateVerificationStatus::UNVERIFIED;
00028     GlobalStateVerificationStatus newStatus = GlobalStateVerificationStatus::UNVERIFIED;
00030     int depth = 0;
00032     StrategyEntry strategy = StrategyEntry();
00034     float stateProbabilityPrevious = 0.0;
00036     ProbabilityCalculationType previousProbabilityEntryType =
      ProbabilityCalculationType::NONE_PROBABILITY;
00038     ProbabilityCalculationType activeProbabilityEntryType =
      ProbabilityCalculationType::SUM_PROBABILITY;
00040     ProbabilityTrueFalse previousProbabilityAnswerValues;
00041     string toString() {
00042         char buff[1024] = { 0 };
00043         if (this->type == HistoryEntryType::DECISION) {
00044             snprintf(buff, sizeof(buff), "decision in %s: to %s", this->globalState->hash.c_str(),
      this->decision->to->hash.c_str());
00045         }
00046         else if (this->type == HistoryEntryType::STATE_STATUS) {
00047             snprintf(buff, sizeof(buff), "stateVerifStatus of %s: %s -> %s",
      this->globalState->hash.c_str(), verifStatusToStr(this->prevStatus).c_str(),
      verifStatusToStr(this->newStatus).c_str());
00048         }
```

```
00049         else if (this->type == HistoryEntryType::CONTEXT) {
00050             snprintf(buff, sizeof(buff), "context in %s: to %s (%s)", this->globalState->hash.c_str(),
      this->decision->to->hash.c_str(), this->globalTransitionControlled ? "controlled" : "uncontrolled");
00051         }
00052         else if (this->type == HistoryEntryType::MARK_DECISION_AS_INVALID) {
00053             snprintf(buff, sizeof(buff), "markInvalid in %s: to %s", this->globalState->hash.c_str(),
      this->decision->to->hash.c_str());
00054         }
00055         else if (this->type == HistoryEntryType::UNCONTROLLED_DECISION) {
00056             snprintf(buff, sizeof(buff), "uncontrolledDecision in %s: to %s (%s)",
      this->globalState->hash.c_str(), this->decision->to->hash.c_str(), this->globalTransitionControlled ?
      "controlled" : "uncontrolled");
00057         }
00058         return string(buff);
00059     };
00060 };
00061
00063 enum GraphTraversalMode {
00064     NORMAL_TRAVERSAL,
00065     RESTORE_STATES,
00066 };
00067
00068
00070 class VerificationIterative {
00071     public:
00072         VerificationIterative(GlobalModelGenerator* generator);
00073         ~VerificationIterative();
00074         map<bitset<STRATEGY_BITS>, string, StrategyBitsComparator> getNaturalStrategy();
00075         vector<tuple<vector<tuple<bool, string>, string>> getReducedStrategy();
00076         int getStrategyComplexity();
00077         Result verify();
00078     protected:
00080         GraphTraversalMode mode = GraphTraversalMode::NORMAL_TRAVERSAL;
00082         GlobalState* revertToGlobalState;
00084         GlobalModelGenerator* generator;
00086         stack<DecisionEntry> history;
00088         map<bitset<STRATEGY_BITS>, string, StrategyBitsComparator> naturalStrategy;
00090         short strategyVariableLimit;
00092         vector<string> variableNames;
00094         int reductionComplexityBefore;
00095
00096         bool verifyLocalStates(vector<LocalState*>* localStates, GlobalState* globalState);
00097         bool isGlobalTransitionControlledByCoalition(GlobalTransition* globalTransition);
00098         bool isAgentInCoalition(Agent* agent);
00099         EpistemicClass* getEpistemicClassForGlobalState(GlobalState* globalState);
00100         bool areGlobalStatesInTheSameEpistemicClass(GlobalState* globalState1, GlobalState*
      globalState2);
00101         string checkIfProbabilistic(GlobalTransition* transitionToCheck);
00102         void findLoopedProbabilityTransitions(map<string, set<GlobalTransition*>>* transitionSets);
00103         void dissolveLoopedProbabilityTransition(GlobalTransition* transitionToDissolve,
      set<GlobalTransition*>* probabilityTransitions);
00104
00105         // todo: fix history
00106         // add and take away from stack the StateVerificationInfo
00107         void addHistoryDecision(GlobalState* globalState, GlobalTransition* decision);
00108         void addHistoryStateStatus(GlobalState* globalState, GlobalStateVerificationStatus prevStatus,
      GlobalStateVerificationStatus newStatus);
00109         bool addHistoryContext(GlobalState* globalState, int depth, GlobalTransition* decision, bool
      globalTransitionControlled);
00110         void addHistoryProbability(GlobalTransition* decision);
00111         void addHistoryAnswerProbability(StateVerificationInfo* globalState,
      ProbabilityCalculationType currentMode, ProbabilityTrueFalse newProbabilityValues);
00112         void addHistoryMarkDecisionAsInvalid(GlobalState* globalState, GlobalTransition* decision);
00113         bool revertToLastDecision();
00114         void undoLastHistoryEntry();
00115
00116         bool equivalentGlobalTransitions(GlobalTransition* globalTransition1, GlobalTransition*
      globalTransition2);
00117         bitset<STRATEGY_BITS> globalStateToValueBits(GlobalState* globalState);
00118         vector<tuple<vector<tuple<bool, string>, string>
      reduceStrategy(vector<tuple<vector<tuple<bool, string>, string>> strategyEntries, short lockedColumn =
      0, bool upperHalf = false);
00119         bool testForAndFixBadAgents(StateVerificationInfo* stateVerificationInfo);
00120         stack<StateVerificationInfo> statesToProcess;
00121 };
00122
00123 #endif // VERIFICATION_ITERATIVE
```

Index